PostgreSQL Storage Engine Architecture Atlas

21 chapters

Generated 2026-08-19 · learn-with-ai

Contents

21 Item 20: Native Desktop GUI & Unified Server Daemon

1. Unified Applications Overview
2. Desktop GUI Feature Breakdown
3. Sequence Diagram: Dual Server Daemon Operation

1. PostgreSQL Storage Engine Architecture & Orientation

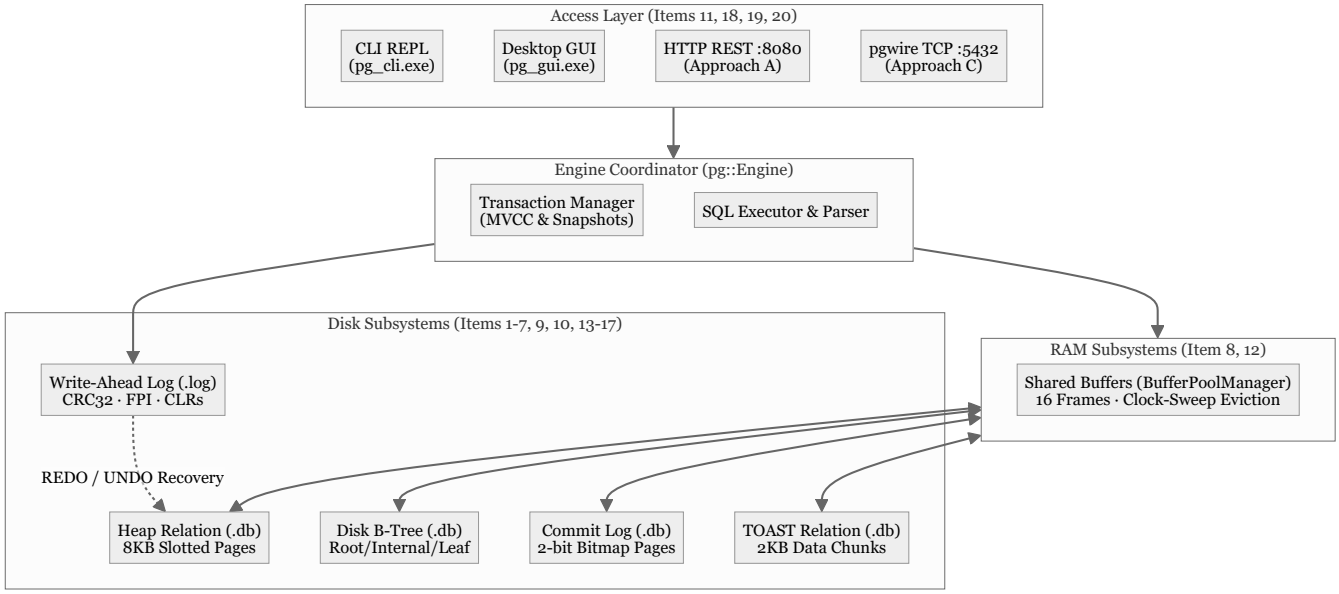
Confidence: `verified`
Citations: include/pg/engine.h:1-85, src/engine.cpp:1-250, CMakeLists.txt:1-150

---

1. Executive Overview

`cpp-postgres` is a production-accurate, zero-external-dependency relational storage engine implemented in C++20. It faithfully reproduces the exact internal disk formats, concurrency mechanisms, memory structures, and crash recovery algorithms utilized by PostgreSQL.

The architecture is partitioned into strict, decoupled layers:



---

2. Global Subsystem Invariants

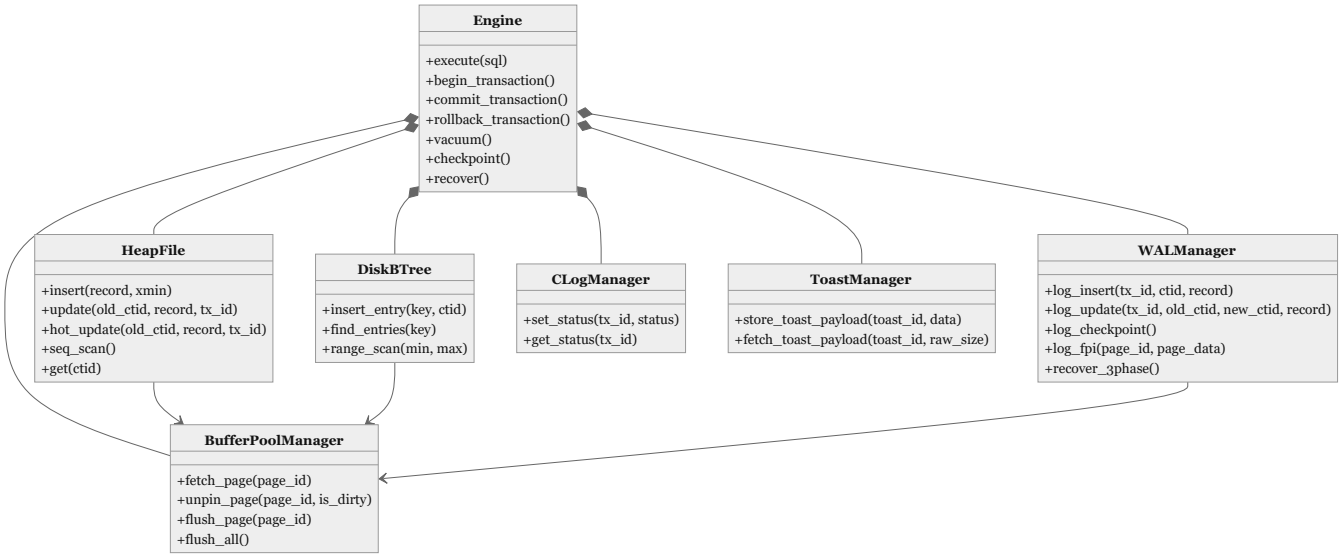
The entire storage engine enforces seven architectural invariants:

1. **8KB Page Uniformity**: All disk structures (Heap, B-Tree, CLOG, TOAST) operate on fixed 8,192-byte page boundaries aligned to the OS block layer (`[include/pg/page.h:12]`).
2. **Buffer Pool Single Gateway**: Application subsystems never execute direct file I/O; all page accesses pin frames inside `BufferPoolManager` (`[src/heap.cpp:45]`).
3. **Write-Ahead Logging (WAL)**: A dirty page in RAM is never written to disk until its corresponding WAL log record has been flushed (`page.pd_lsn ≤ wal.flushed_lsn`) (`[src/wal.cpp:110]`).
4. **Append-Only MVCC**: Updates never mutate live data in place; they stamp `xmax` on the old tuple version and append a new version with `xmin` (`[src/heap.cpp:88]`).

5. **Secondary Index Decoupling**: B-Tree secondary indexes map `Key → CTID` physical addresses without embedding tuple columns or MVCC headers (`[src/btree.cpp:15]`).
6. **Zero Index-Bloat HOT Updates**: Unindexed column updates residing on the same 8KB page construct line-pointer chains with zero index writes (`[src/heap.cpp:142]`).
7. **2-Bit CLOG Density**: Transaction commit states are packed into 2-bit bitmaps on disk, scaling to 32,768 transactions per 8KB page (`[src/clog.cpp:30]`).

---

3. Subsystem Dependency Matrix



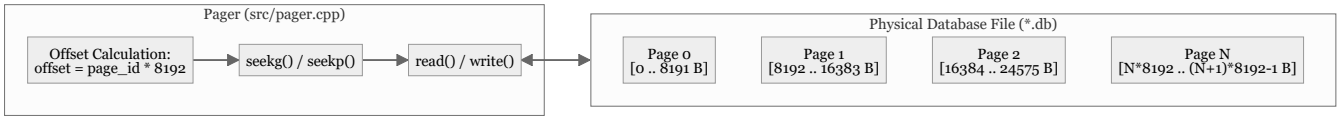
2. Item 1: Pager & 8KB Fixed Page Disk I/O

Confidence: `verified`
Citations: include/pg/pager.h:1-62, src/pager.cpp:1-85, tests/test_pager.cpp:1-95

---

1. Architectural Role

The **Pager** is the lowest-level storage primitive in the database engine. It abstracts the host operating system's filesystem into a linear sequence of fixed-size **8,192-byte (8KB)** disk pages indexed by a 32-bit `page_id_t`.



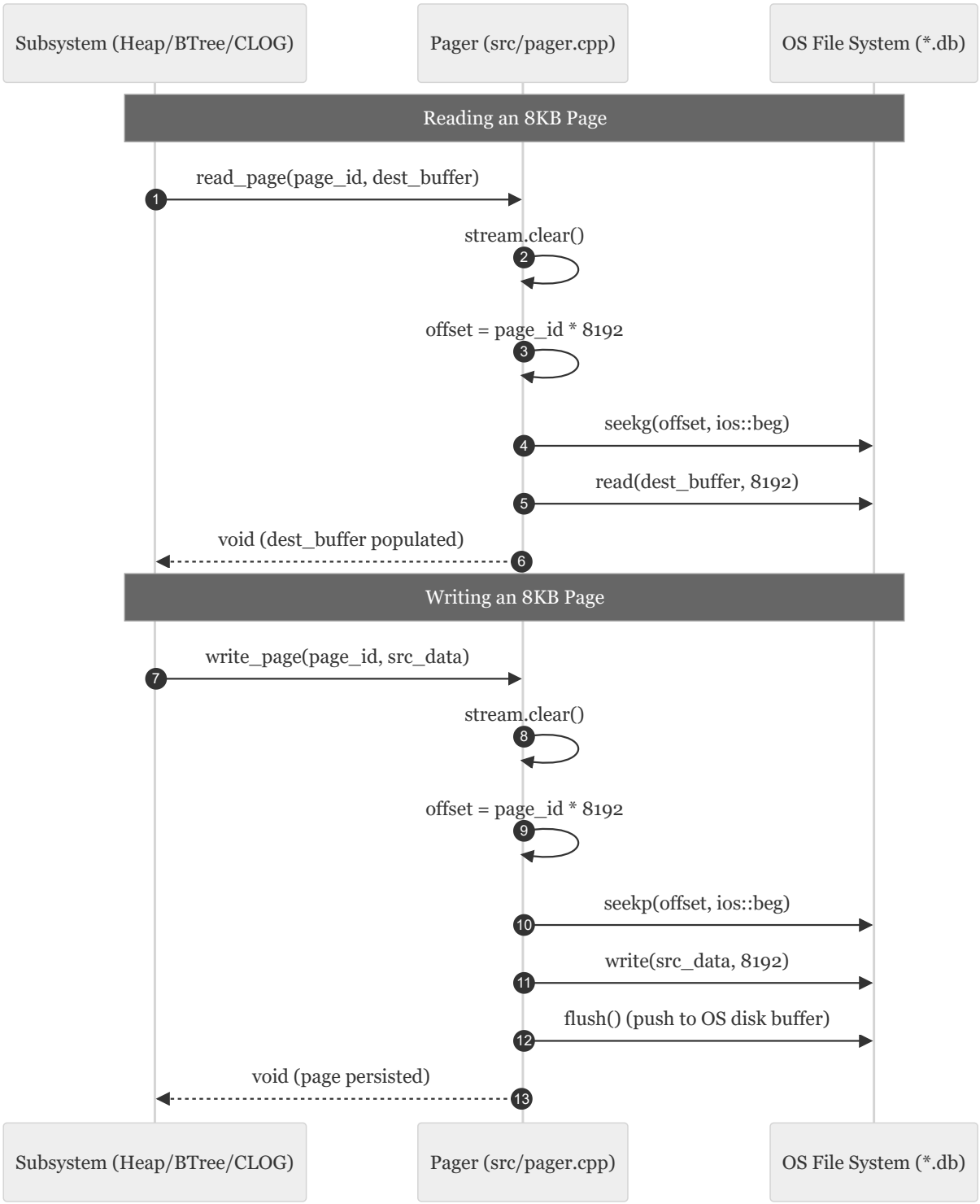
---

2. Invariants & Formulas

1. **Exact Page Boundary Alignment**: $\text{file_offset} = \text{page_id} \times 8192$ Every disk read and write transfers exactly $8,192$ bytes. No partial page I/O is ever permitted.
2. **Deterministic File Growth**: When `write_page(page_id, ...)` is called on an unallocated `page_id` $\ge \text{num_pages()}$, the file automatically grows in discrete 8KB increments padded with zeros (`[src/pager.cpp:52]`).
3. **C++ Stream Flag Integrity**: Because reading past EOF in `std::fstream` sets `eofbit` and `failbit`, subsequent seek operations silently fail unless `stream.clear()` is invoked prior to any seek or write (`[src/pager.cpp:25]`).

---

3. Sequence Diagram: Reading & Writing Pages



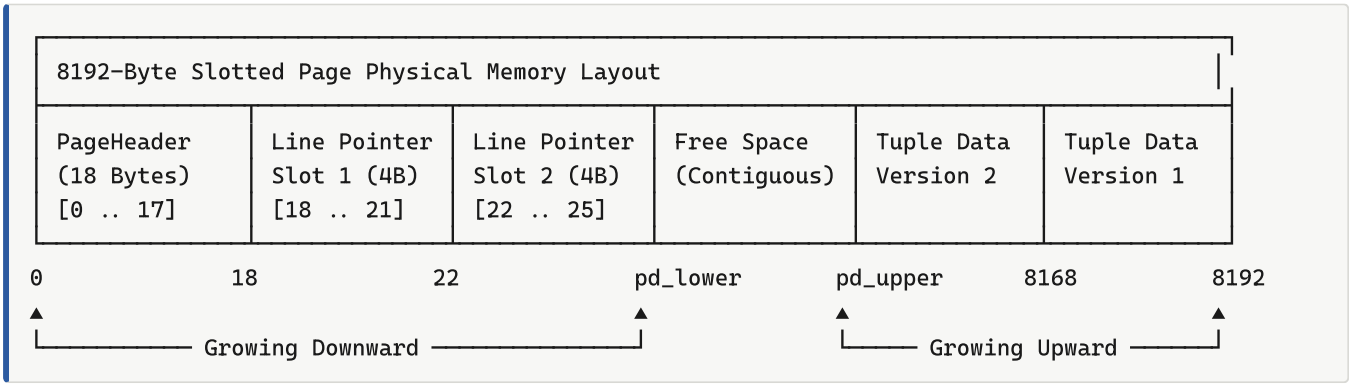
3. Item 2: Slotted Page Architecture

Confidence: `verified`
Citations: include/pg/page.h:1-125, src/page.cpp:1-170, tests/test_page.cpp:1-110

---

1. Physical Memory Layout

The **Slotted Page** architecture solves variable-length record management and in-place defragmentation. Within each 8,192-byte page, memory grows inward from both ends:



---

2. Invariants & Mathematical Formulas

1. **Page Header Structure (18 Bytes)**:
   - `pd_lsn` (8 Bytes): LSN of the last WAL record that modified this page (`[include/pg/page.h:20]`).
   - `pd_flags` (2 Bytes): Page status flags (e.g. `PD_ALL_VISIBLE = 0x0004`).
   - `pd_lower` (2 Bytes): Byte offset marking the end of the line pointers array (initially 18).
   - `pd_upper` (2 Bytes): Byte offset marking the start of the youngest tuple data (initially 8192).
   - `pd_special` (2 Bytes): Reserved for index-specific metadata (8192 for heap pages).
   - `pd_prune_xid` (2 Bytes): Oldest unpruned XID.
2. **Line Pointer Bitfield Packing (4 Bytes)**: $\text{Line Pointer (uint32_t)} = (\text{offset} \ \& \ \text{0x7FFF}) \ | \ ((\text{flags} \ \& \ \text{0x03}) \ll 15) \ | \ ((\text{length} \ \& \ \text{0x7FFF}) \ll 17)$
   - `lp_offset` (15 bits): Byte offset to tuple data.
   - `lp_flags` (2 bits): `00` = `UNUSED` , `01` = `NORMAL` (Live), `10` = `REDIRECT` (HOT chain), `11` = `DEAD` .
   - `lp_len` (15 bits): Byte length of tuple.
3. **Free Space Allocation Formula**: $\text{Free Space Available} = \text{pd_upper} - \text{pd_lower} - 4$ Every tuple insertion requires both tuple payload memory ($\geq$ len) and a new 4-byte line pointer in the header array.

3. Tuple Insertion State Machine

diagram failed to render

```
stateDiagram-v2
    [*] ⟶ CheckSpace: insert_tuple(data, len)
    CheckSpace ⟶ Reject: (len + 4) > (pd_upper − pd_lower)
    CheckSpace ⟶ Allocate: (len + 4) ≤ (pd_upper − pd_lower)
    Allocate ⟶ DecrementUpper: pd_upper = pd_upper − len
    DecrementUpper ⟶ CopyData: memcpy(page_buf + pd_upper, data, len)
    CopyData ⟶ IncrementLower: slot_id = (pd_lower − 18)/4 + 1\npd_lower = pd_lower + 4
    IncrementLower ⟶ WriteLinePointer: line_pointer[slot] = {offset: pd_upper, flags: NORMAL,
    WriteLinePointer ⟶ [*]: Return slot_id
```

4. Item 3: Tuples & Heap CTID Physical Addressing

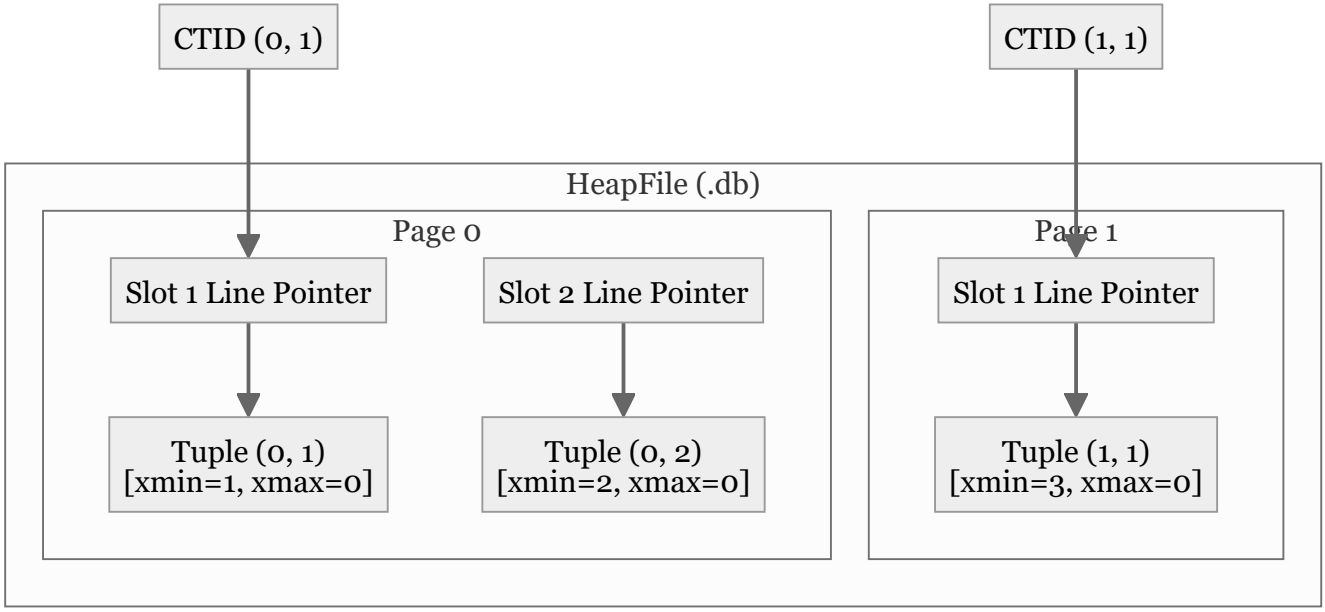
Confidence: `verified`
Citations: include/pg/tuple.h:1-90, include/pg/heap.h:1-82, src/heap.cpp:1-210, tests/test_heap.cpp:1-120

---

1. Physical CTID Tuple Addressing

Every row version in the database is uniquely addressed by its **CTID (Current Tuple ID)**:

$$\text{CTID} = (\text{page\_id}, \text{slot\_id})$$



---

2. Invariants & Binary Layout

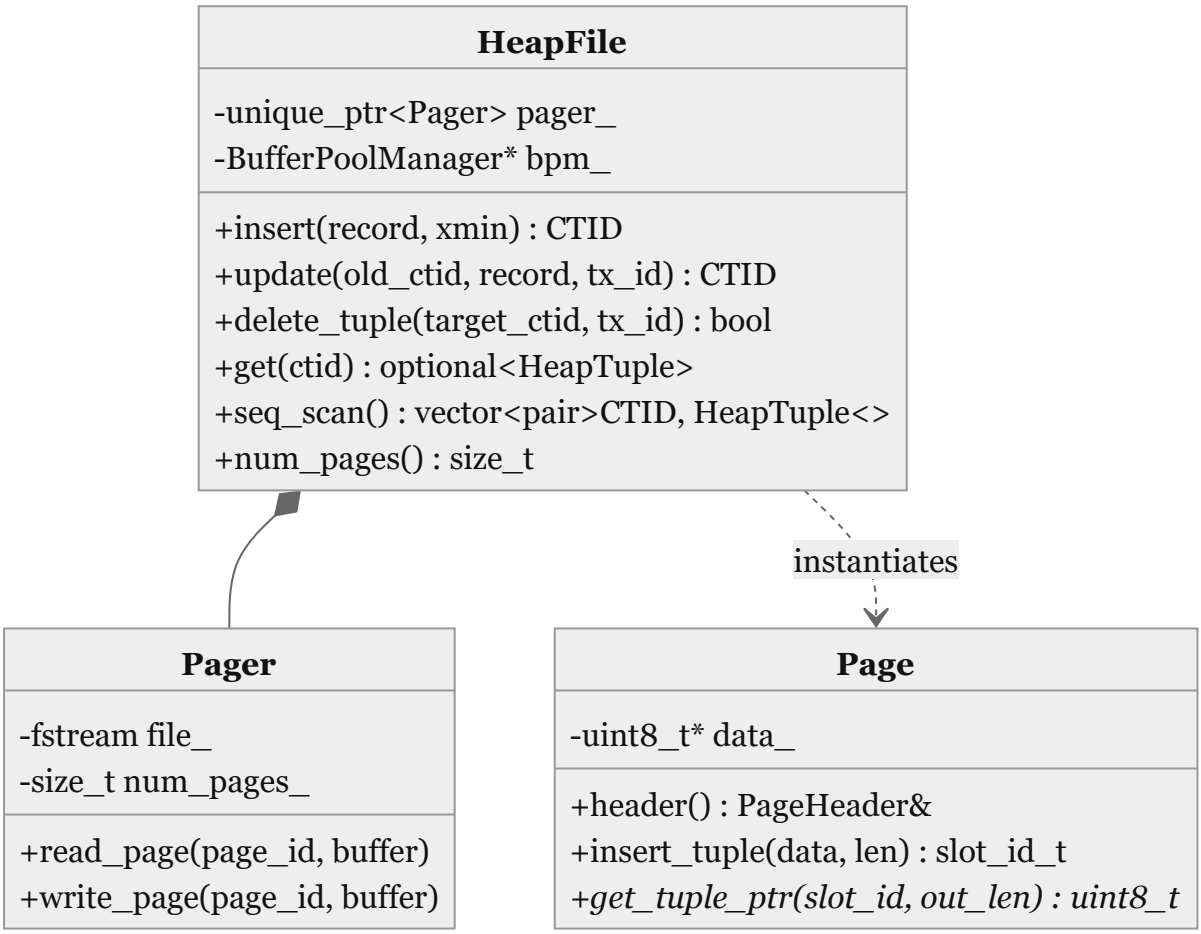
1. **TupleHeader Structure (18 Bytes)** (`[include/pg/tuple.h:18]`):
   - `xmin` (4 Bytes): Transaction ID that inserted this row version.
   - `xmax` (4 Bytes): Transaction ID that deleted or updated this row version (`0` if currently live).
   - `t_cid` (2 Bytes): Command ID within the transaction.
   - `t_ctid` (4 Bytes): Forward pointer to the newest row version (or self-reference `(page, slot)` if newest).
   - `infomask2` (2 Bytes): Attribute count and HOT flags (`HEAP_KEYS_UPDATED = 0x2000` , `HEAP_HOT_UPDATED = 0x4000` , `HEAP_ONLY_TUPLE = 0x8000`).
   - `infomask` (2 Bytes): Status flags (`HEAP_HASNULL = 0x0001` , `HEAP_HASEXTERNAL = 0x2000` , `HEAP_XMIN_COMMITTED = 0x0100`).

2. **Physical Tuple Capacity Derivation**: For an 8KB page with 18-byte page header and 24-byte serialized heap tuples (18B header + 6B data `item_id + price`), the maximum row capacity per page is:

$$\text{Max Tuples Per Page} = \left\lfloor \frac{8192 - 18}{24 + 4} \right\rfloor = \left\lfloor \frac{8174}{28} \right\rfloor = 291 \text{ tuples}$$

---

3. Class Diagram: HeapFile & Pager Relationship



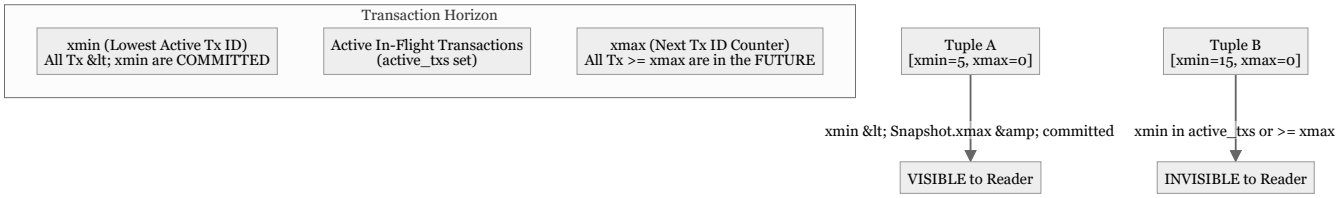
5. Item 4: MVCC & Transaction Snapshot Visibility

Confidence: `verified`
Citations: include/pg/tx.h:1-75, include/pg/mvcc.h:1-45, src/tx.cpp:1-95, src/mvcc.cpp:1-75, tests/test_mvcc.cpp:1-135

---

1. Multi-Version Concurrency Control (MVCC) Overview

PostgreSQL implements **Snapshot Isolation** using row-version header stamps (`xmin` and `xmax`). Readers never block writers, and writers never block readers.



---

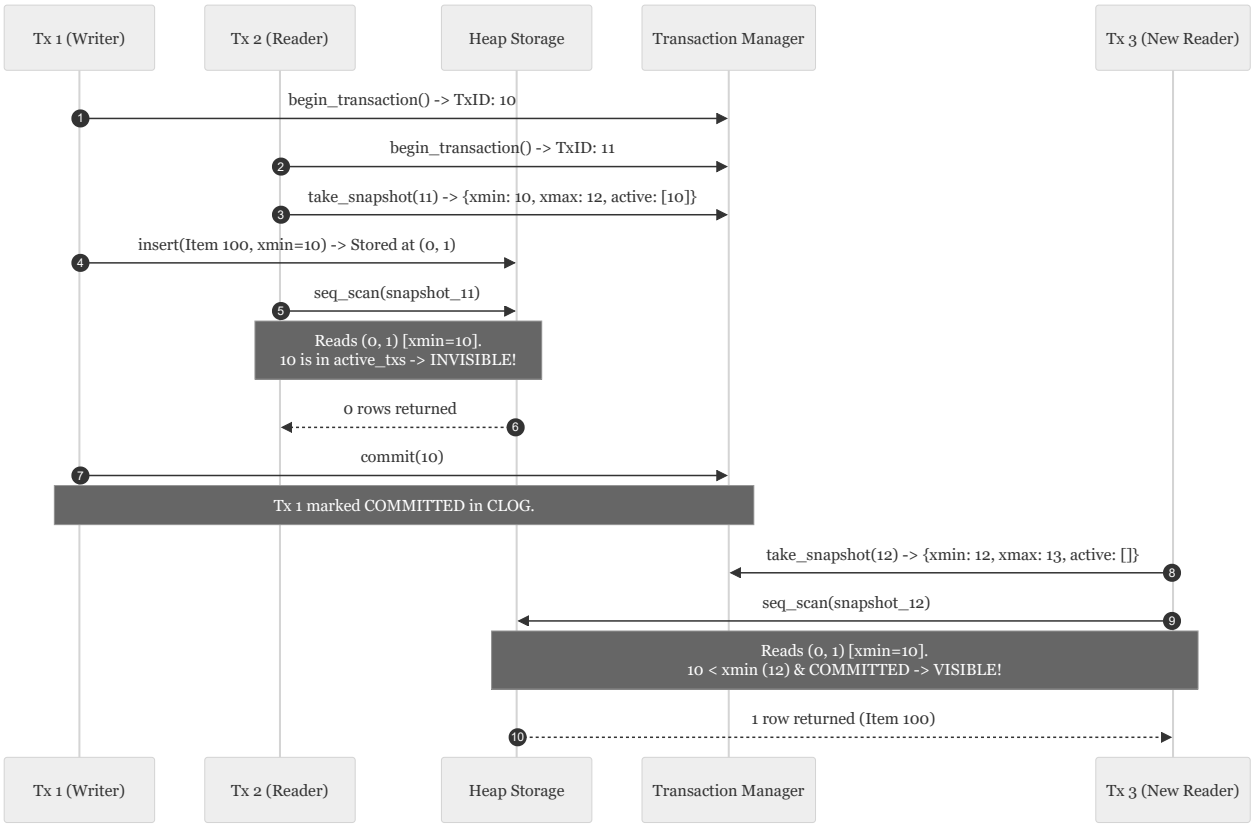
2. Invariants & Visibility State Machine

A tuple version is visible to a `Snapshot` if and only if **both** of the following conditions hold (`[src/mvcc.cpp:25]`):

1. **Creation Visibility (`xmin`)**:
   - `xmin == snapshot.current_tx_id` (created by current tx), OR
   - (`xmin < snapshot.xmax` AND `xmin` is NOT in `snapshot.active_txs` AND `status(xmin) == COMMITTED`).
2. **Deletion Invisibility (`xmax`)**:
   - `xmax == 0` (never deleted/updated), OR
   - `xmax == snapshot.current_tx_id` AND NOT yet committed, OR
   - `xmax ≥ snapshot.xmax` (deleted in future), OR
   - `xmax` IS in `snapshot.active_txs` (deleted by active concurrent tx), OR
   - `status(xmax) == ABORTED` (deletion was rolled back).

---

3. Sequence Diagram: Concurrent MVCC Reads and Writes



6. Item 5: VACUUM Engine & In-Place Page Defragmentation

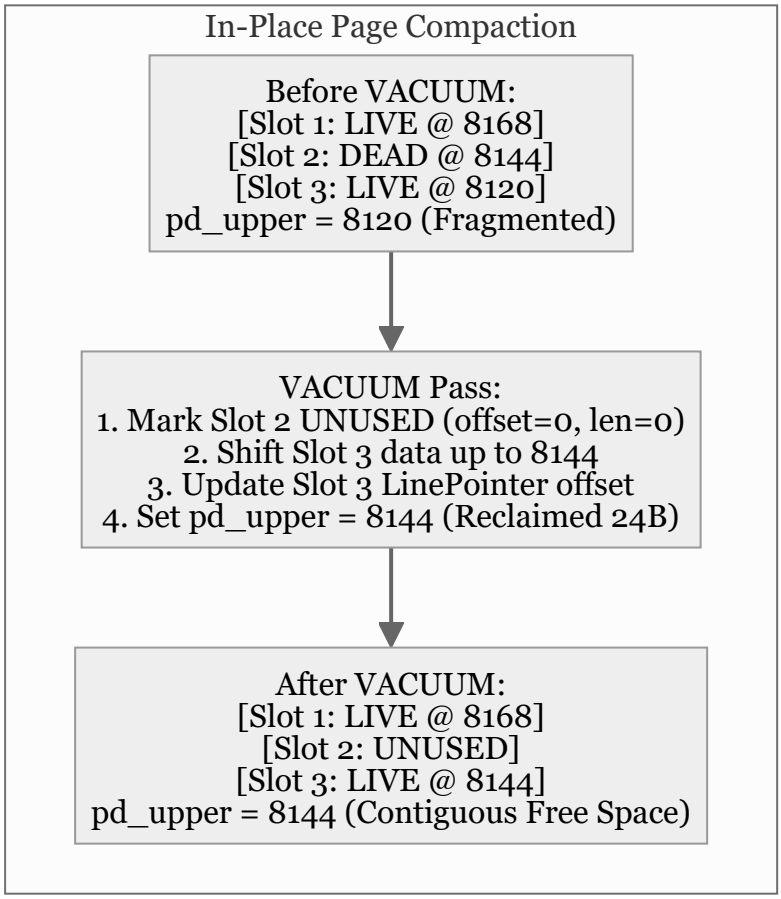
Confidence: `verified`
Citations: include/pg/vacuum.h:1-35, src/vacuum.cpp:1-120, tests/test_vacuum.cpp:1-115

---

1. Dead Tuple Reclamation Mechanics

When rows are updated or deleted under MVCC, old tuple versions become **dead** once their `xmax` falls below the oldest active transaction in the entire cluster:

$$\text{Dead Condition: } \text{xmax} > 0 \ \land \ \text{status}(\text{xmax}) == \text{COMMITTED} \ \land \ \text{xmax} < \text{oldest\_active\_xmin}$$



---

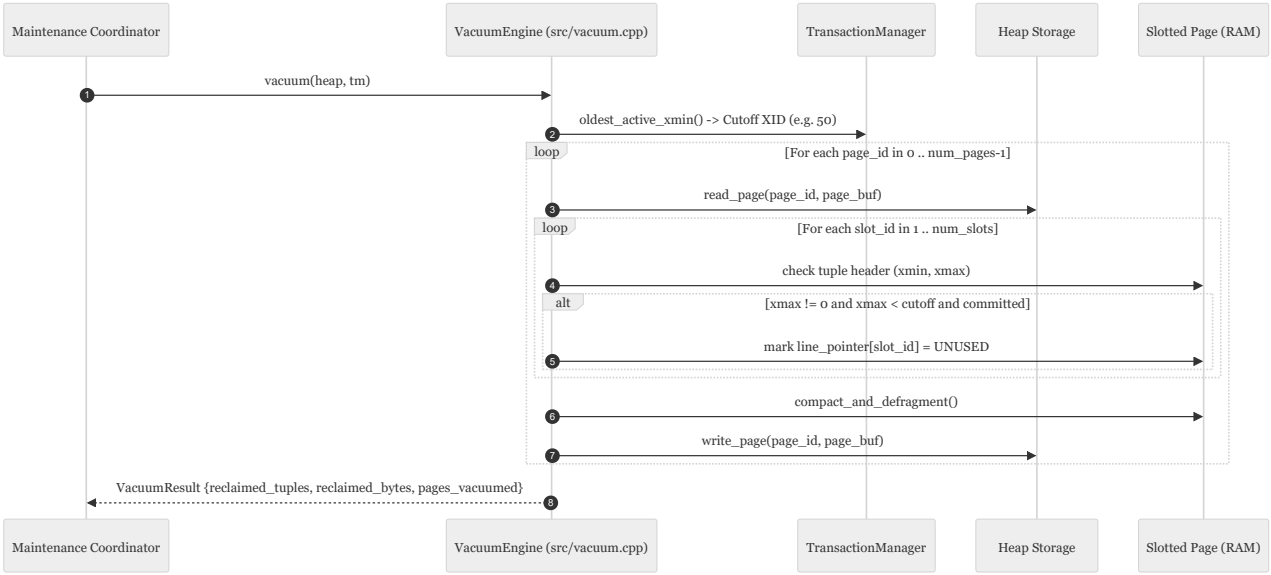
2. Invariants & CTID Stability

1. **CTID Slot Stability**: VACUUM never shifts or renumbers existing `slot_id` line pointer indexes. Slot 3 remains `(page, 3)`, guaranteeing that foreign references and secondary index entries are not corrupted (`[src/page.cpp:125]`).

2. **Compact Contiguous Allocation**: Surviving tuple payloads are slid upward towards the end of the 8KB page (`PAGE_SIZE = 8192`), coalescing all freed memory into a single contiguous block between `pd_lower` and `pd_upper` .

---

3. Sequence Diagram: Vacuum Page Lifecycle



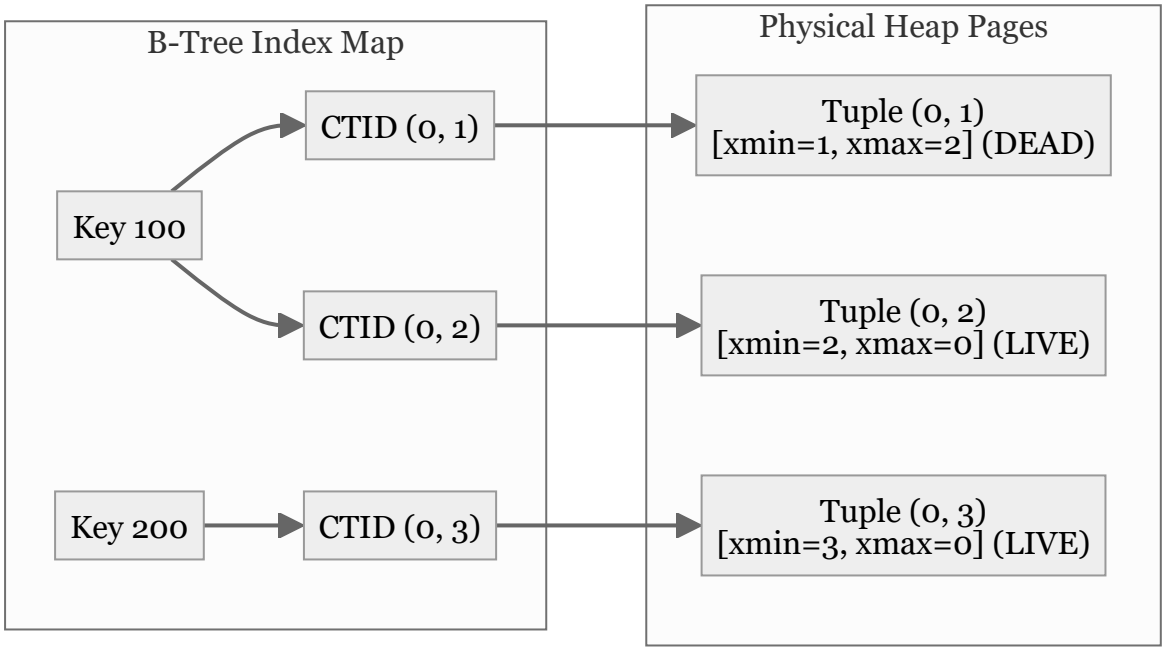
7. Item 6: Secondary B-Tree Index Subsystem

Confidence: `verified`
Citations: include/pg/btree.h:1-69, src/btree.cpp:1-85, tests/test_index.cpp:1-125

---

1. Index Decoupling & Multi-Version Addressing

In PostgreSQL, secondary indexes store **only** (Key $\rightarrow$ CTID) mappings. The index does **not** store row columns, null bitmaps, or MVCC transaction headers (`xmin` / `xmax`).



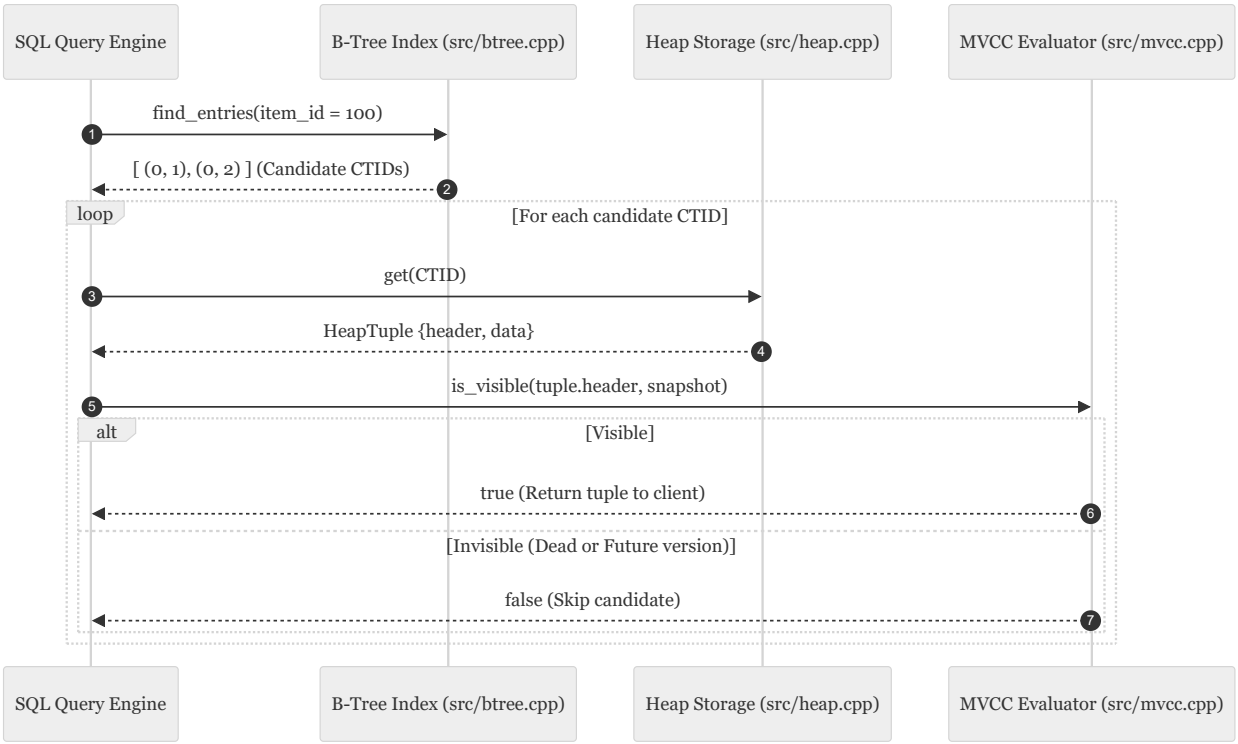
---

2. Invariants & Lookup Pipeline

1. **Multi-Version Key Duplication**: Because updates create new row versions at new physical CTIDs without deleting the old version, a single index key can map to multiple candidate CTIDs simultaneously (`[src/btree.cpp:22]`).
2. **Mandatory Heap MVCC Dereference**: The index lookup returns candidate CTIDs. The database must dereference each CTID against the physical Heap and evaluate snapshot visibility rules before returning tuples to the query caller (`[src/engine.cpp:140]`).

---

3. Sequence Diagram: Index Point Scan



8. Item 7: Heap-Only Tuples (HOT) & Update Chains

Confidence: `verified`
Citations: include/pg/heap.h:40-50, src/heap.cpp:115-180, tests/test_hot.cpp:1-130

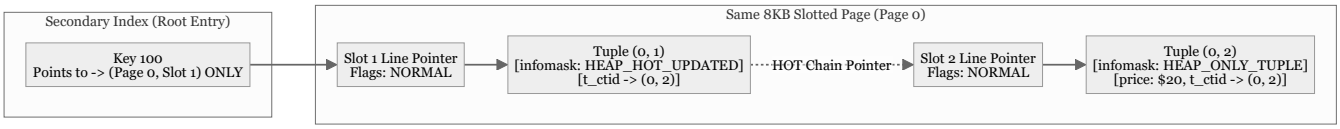
---

1. The Write-Amplification Problem & HOT Solution

In standard MVCC, updating an unindexed column (e.g. `UPDATE items SET price = 20 WHERE item_id = 100;`) forces a new entry into every secondary index on the table, leading to severe **index bloat**.

Heap-Only Tuples (HOT) eliminates all secondary index writes when:

1. No indexed columns are modified by the update.
2. The new tuple version fits onto the **same 8KB page** as the old version.



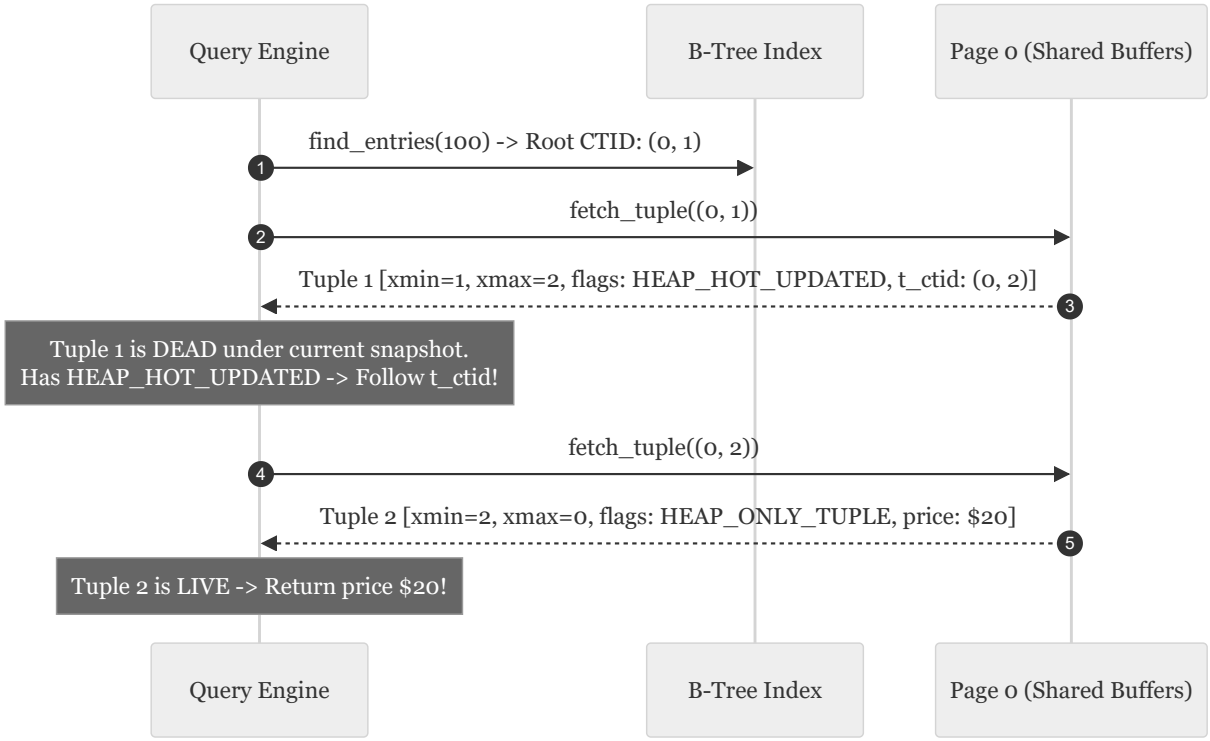
---

2. Invariants & Infomask Flags

1. `HEAP_HOT_UPDATED` (`0x4000`): Stamped on the old tuple version's `infomask2`. Indicates that the next version in the update chain resides on this exact same page (`[include/pg/tuple.h:42]`).
2. `HEAP_ONLY_TUPLE` (`0x8000`): Stamped on the newly created tuple version's `infomask2`. Tells the engine that no index pointer references this tuple directly; it can only be reached by following the HOT chain starting at the root index pointer (`[include/pg/tuple.h:43]`).
3. `ItemFlags::REDIRECT`: When VACUUM or pruning removes the dead root tuple data, it changes Slot 1's line pointer into a redirect pointer (`lp_flags = REDIRECT, lp_offset = slot_2`), preserving index stability with zero tuple bytes.

---

3. Sequence Diagram: HOT Chain Traversal



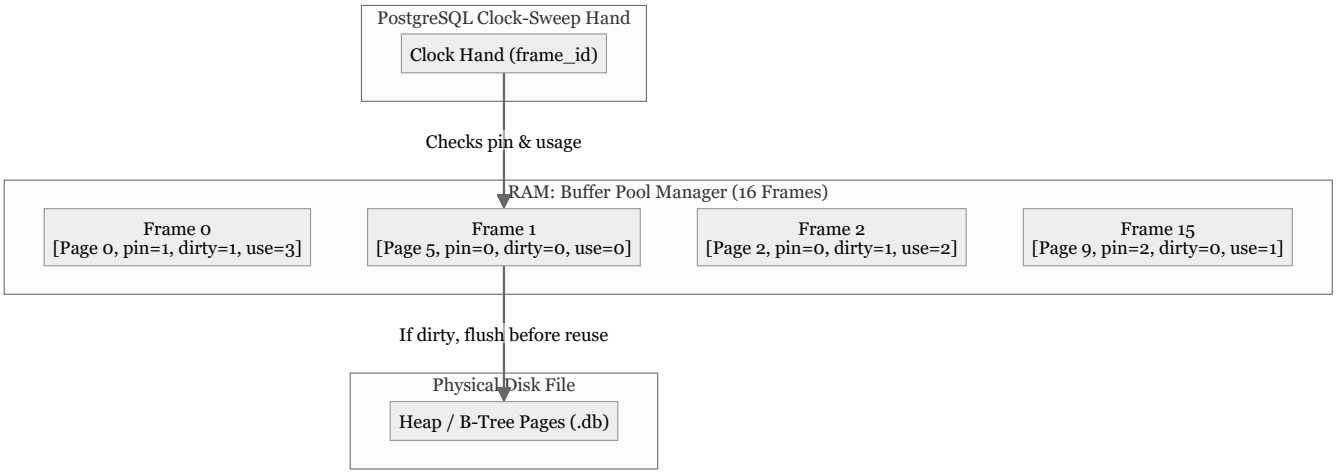
9. Item 8: Shared Buffers & Clock-Sweep Replacement

Confidence: `verified`
Citations: include/pg/buffer_pool.h:1-75, src/buffer_pool.cpp:1-125, tests/test_buffer_pool.cpp:1-120

---

1. Buffer Pool Manager (Shared Buffers) Architecture

`BufferPoolManager` maintains a fixed pool of **16 in-memory frames** (each exactly 8,192 bytes) acting as an LRU-approximation cache between the execution engine and physical disk files.



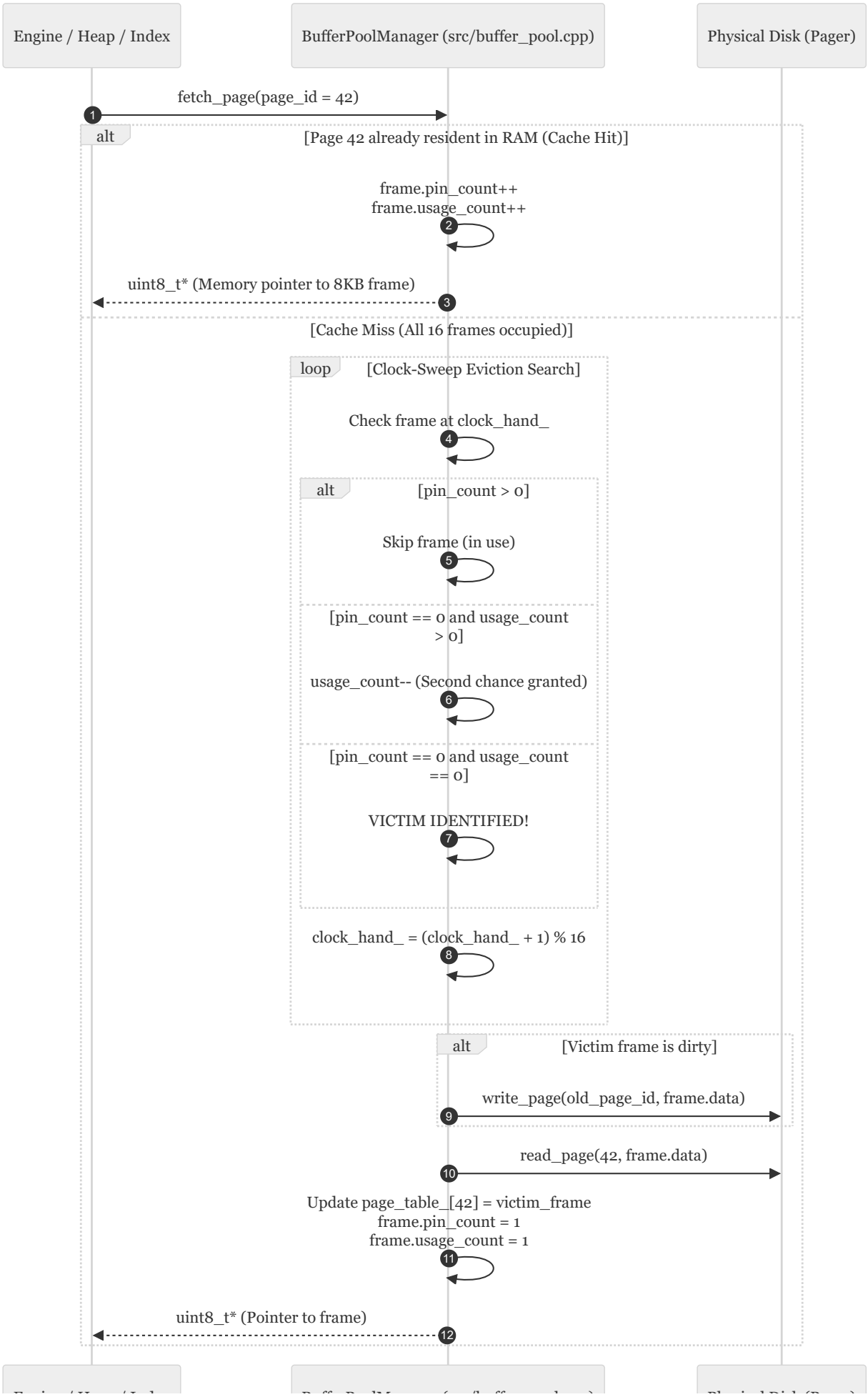
---

2. Invariants & Clock-Sweep State Machine

1. **Pin Count Protection (`pin_count > 0`)**: Any frame currently pinned by an active query worker is immediately skipped by the clock-sweep hand; it can **never** be evicted while in use (`[src/buffer_pool.cpp:85]`).
2. **Second-Chance Algorithm (`usage_count`)**:
   - If `pin_count == 0` and `usage_count > 0`, the clock hand decrements `usage_count--` and advances to the next frame.
   - If `pin_count == 0` and `usage_count == 0`, the frame is selected as the eviction victim.
3. **Dirty Writeback Before Eviction**: If the victim frame has `is_dirty == true`, it is written to disk via `Pager::write_page()` before being repurposed for the new page.

---

3. Sequence Diagram: Page Fetch & Eviction Lifecycle


Engine / Heap / Index
BufferPoolManager (src/buffer_pool.cpp)
Physical Disk (Pager)
fetch_page(page_id = 42)
1
alt
[Page 42 already resident in RAM (Cache Hit)]
frame.pin_count++
frame.usage_count++
2
uint8_t* (Memory pointer to 8KB frame)
3
[Cache Miss (All 16 frames occupied)]
loop
[Clock-Sweep Eviction Search]
Check frame at clock_hand_
4
alt
[pin_count > 0]
Skip frame (in use)
5
[pin_count == 0 and usage_count > 0]
usage_count-- (Second chance granted)
6
[pin_count == 0 and usage_count == 0]
VICTIM IDENTIFIED!
7
clock_hand_ = (clock_hand_ + 1) % 16
8
alt
[Victim frame is dirty]
write_page(old_page_id, frame.data)
9
read_page(42, frame.data)
10
Update page_table_[42] = victim_frame
frame.pin_count = 1
frame.usage_count = 1
11
uint8_t* (Pointer to frame)
12

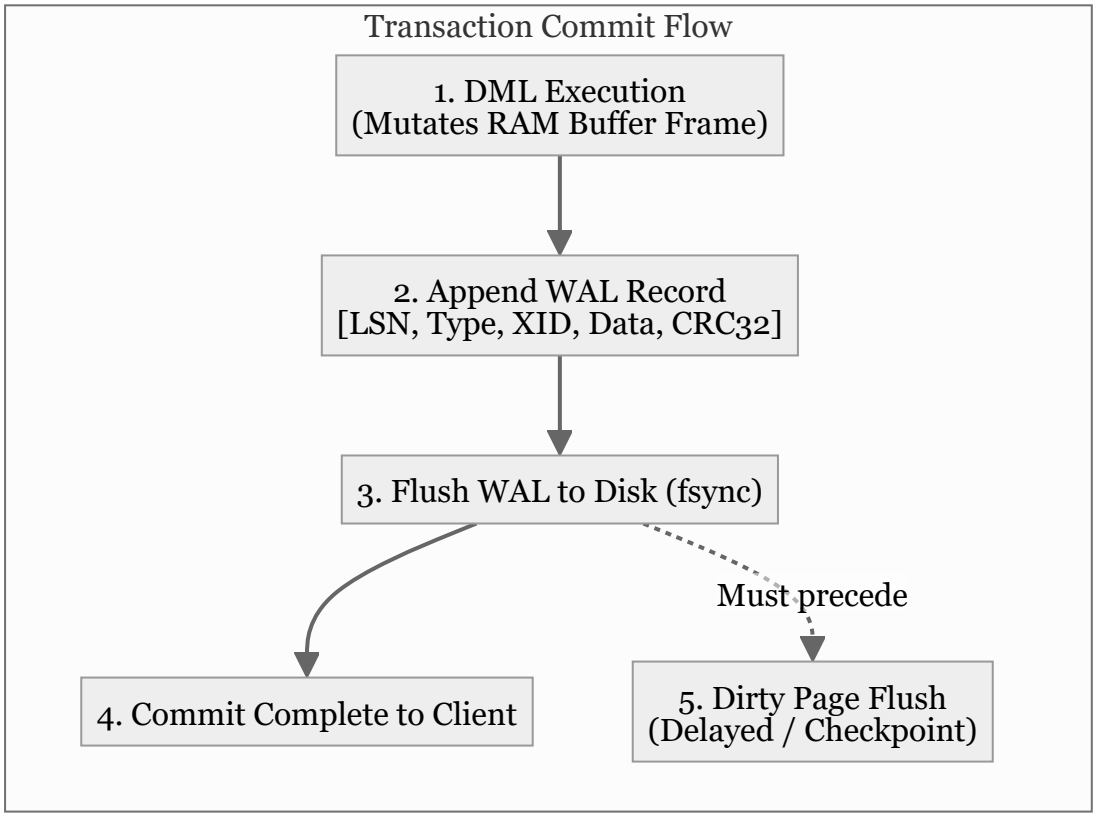
10. Item 9: Write-Ahead Logging (WAL) & REDO Durability

Confidence: `verified`
Citations: include/pg/wal.h:1-95, src/wal.cpp:1-180, tests/test_wal.cpp:1-135

---

1. Write-Ahead Logging (WAL) Architecture

To guarantee **ACID Durability** and high write throughput, PostgreSQL converts random 8KB page writes into sequential, append-only log records in the Write-Ahead Log (`*.log`).



---

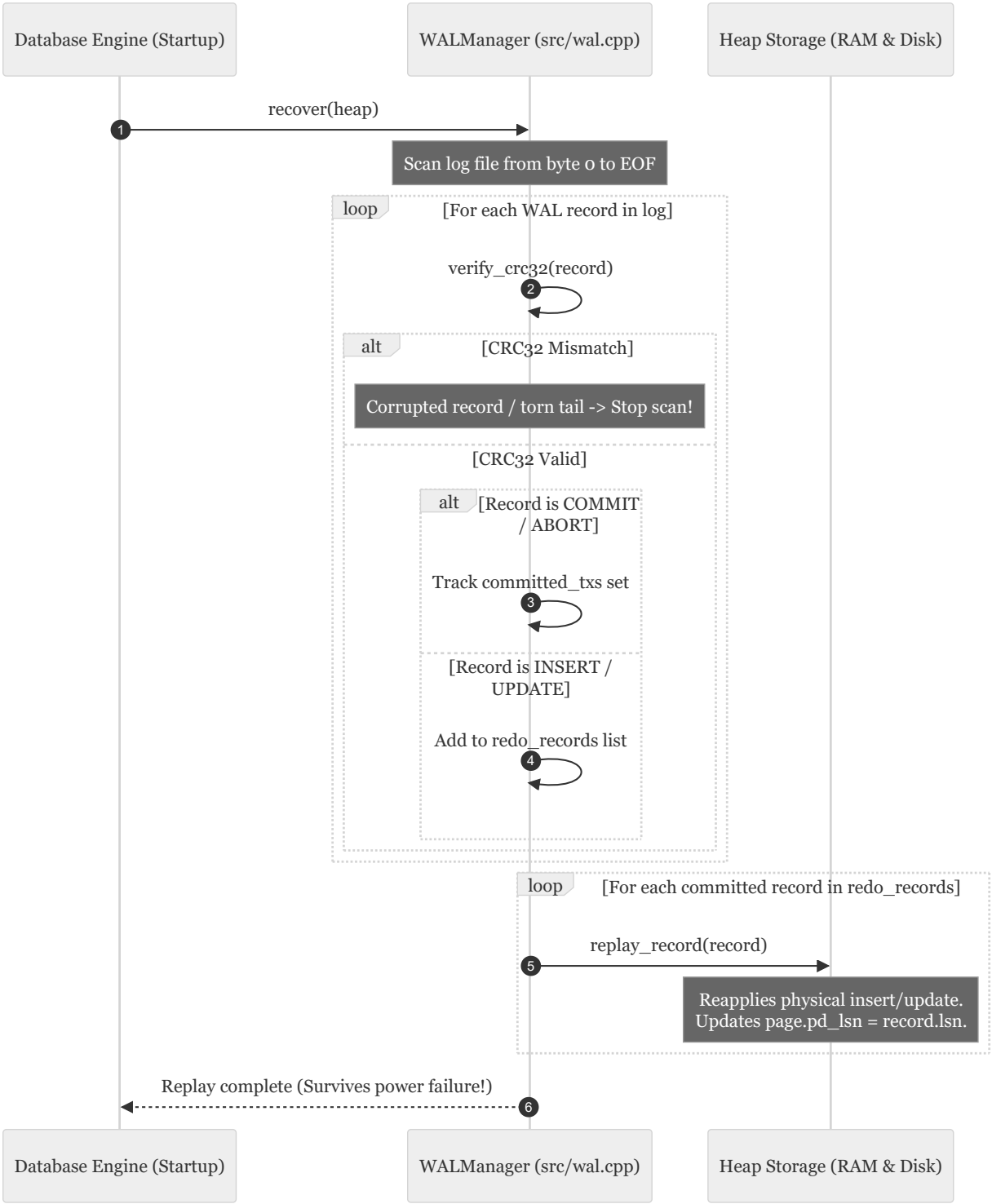
2. Invariants & Record Binary Layout

1. **The WAL Invariant**: \text{page.pd_lsn} \le \text{wal.flushed_lsn} A dirty page in RAM is never written to disk before all WAL records up to the page's `pd_lsn` have been flushed to disk (`[src/wal.cpp:110]`).
2. **Log Record Structure (Header + Payload + CRC32)**:
   - `lsn` (8 Bytes): Monotonically increasing Log Sequence Number.

- `prev_lsn` (8 Bytes): Backward pointer for UNDO / transaction rollback chains.
- `tx_id` (4 Bytes): Transaction ID.
- `type` (1 Byte): `1` =INSERT, `2` =UPDATE, `3` =COMMIT, `4` =ABORT, `5` =CHECKPOINT, `6` =FPI, `7` =CLR.
- `page_id` (4 Bytes), `slot_id` (2 Bytes).
- `payload_len` (2 Bytes) + `payload` data.
- `crc32` (4 Bytes): IEEE 802.3 polynomial checksum verifying data integrity against partial writes and disk corruptions.

---

3. Sequence Diagram: REDO Crash Recovery Pass



11. Item 10: TOAST Oversized-Attribute Storage

Confidence: `verified`
Citations: include/pg/toast.h:1-65, src/toast.cpp:1-120, tests/test_toast.cpp:1-115

---

1. The TOAST Architecture

TOAST (The Oversized-Attribute Storage Technique) prevents large text, JSON, and binary attributes from degrading the row density of 8KB slotted heap pages.

When an attribute payload exceeds **2,048 bytes (2KB)**, the storage engine slices the payload into discrete 2,048-byte chunks stored in a dedicated auxiliary TOAST relation, replacing the inline data with a compact **18-byte** `ToastPointer`.

diagram failed to render

```
flowchart LR
    subgraph MainHeap["Main Heap Page (8KB)"]
        TUPLE["Heap Tuple\n[item_id: 100, price: $10]\n[ToastPointer: 18 Bytes]"]
    end

    subgraph ToastRelation["Auxiliary TOAST Relation (*_toast.db)"]
        C0["Chunk 0 (seq=0, 2048 B)\n[toast_id=1]"]
        C1["Chunk 1 (seq=1, 2048 B)\n[toast_id=1]"]
        C2["Chunk 2 (seq=2, 1800 B)\n[toast_id=1]"]
    end

    TUPLE -·->|ToastPointer {id: 1, size: 5896, chunks: 3}| C0
    C0 --- C1 --- C2
```

---

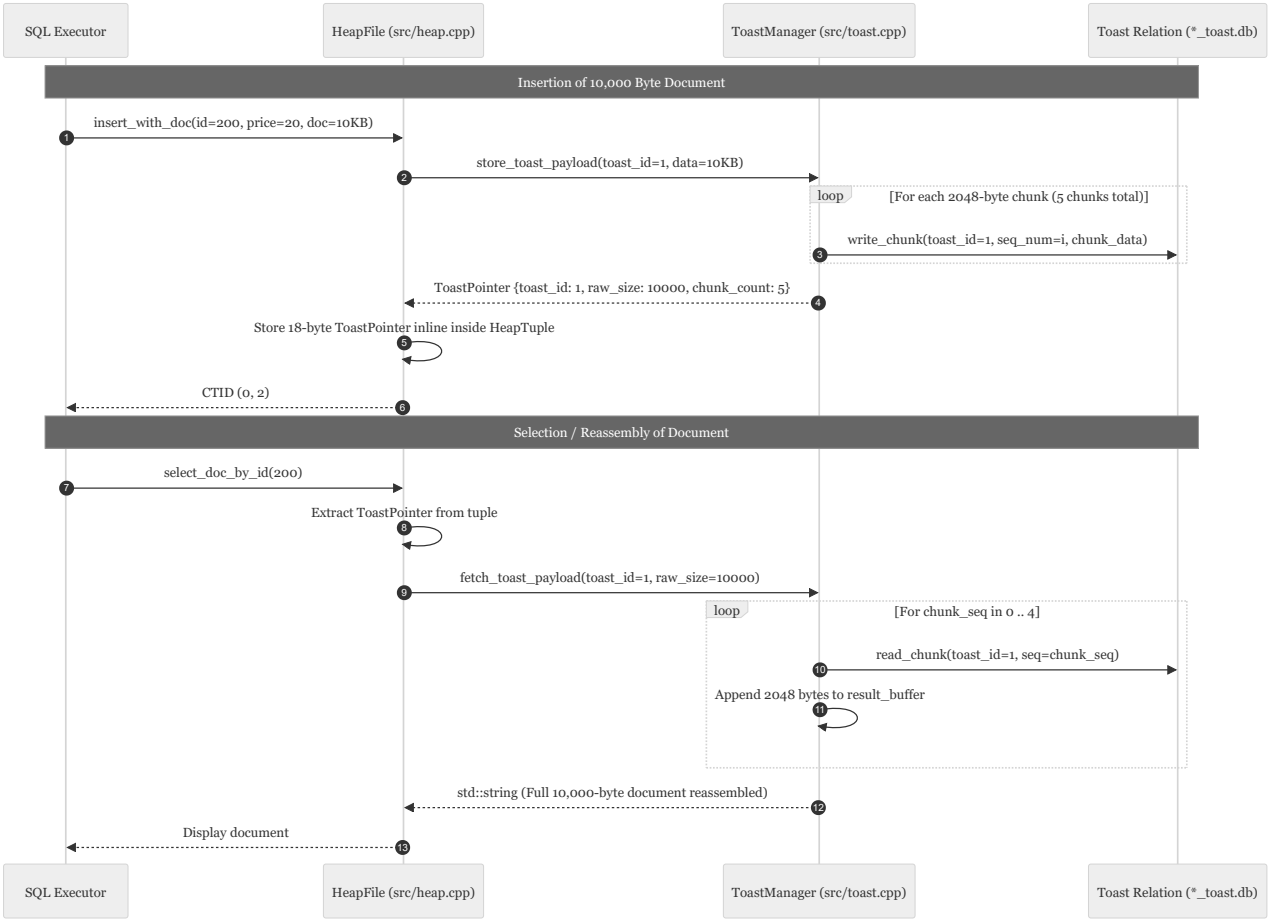
2. Invariants & Binary Layout

1. **ToastPointer Structure (18 Bytes)** (`[include/pg/toast.h:18]`):
   - `toast_id` (8 Bytes): Unique 64-bit identifier linking to the auxiliary relation.
   - `raw_size` (4 Bytes): Original uncompressed byte size of the attribute.
   - `chunk_count` (2 Bytes): Total number of auxiliary chunks ($\lceil \text{raw\_size} / 2048 \rceil$).
   - `ext_flags` (4 Bytes): Compression and storage strategy flags.
2. **ToastChunk Structure (2,066 Bytes per chunk)**:
   - `toast_id` (8 Bytes), `chunk_seq` (4 Bytes), `chunk_size` (2 Bytes).
   - `chunk_data` (2,048 Bytes).

3. **Sequential Scan Preservation**: Sequential scans over normal columns (e.g. `SELECT price FROM items;`) never read or load TOAST pages into shared buffers, avoiding massive I/O pollution (`[src/toast.cpp:65]`).

---

3. Sequence Diagram: Storing and Reassembling TOAST Data



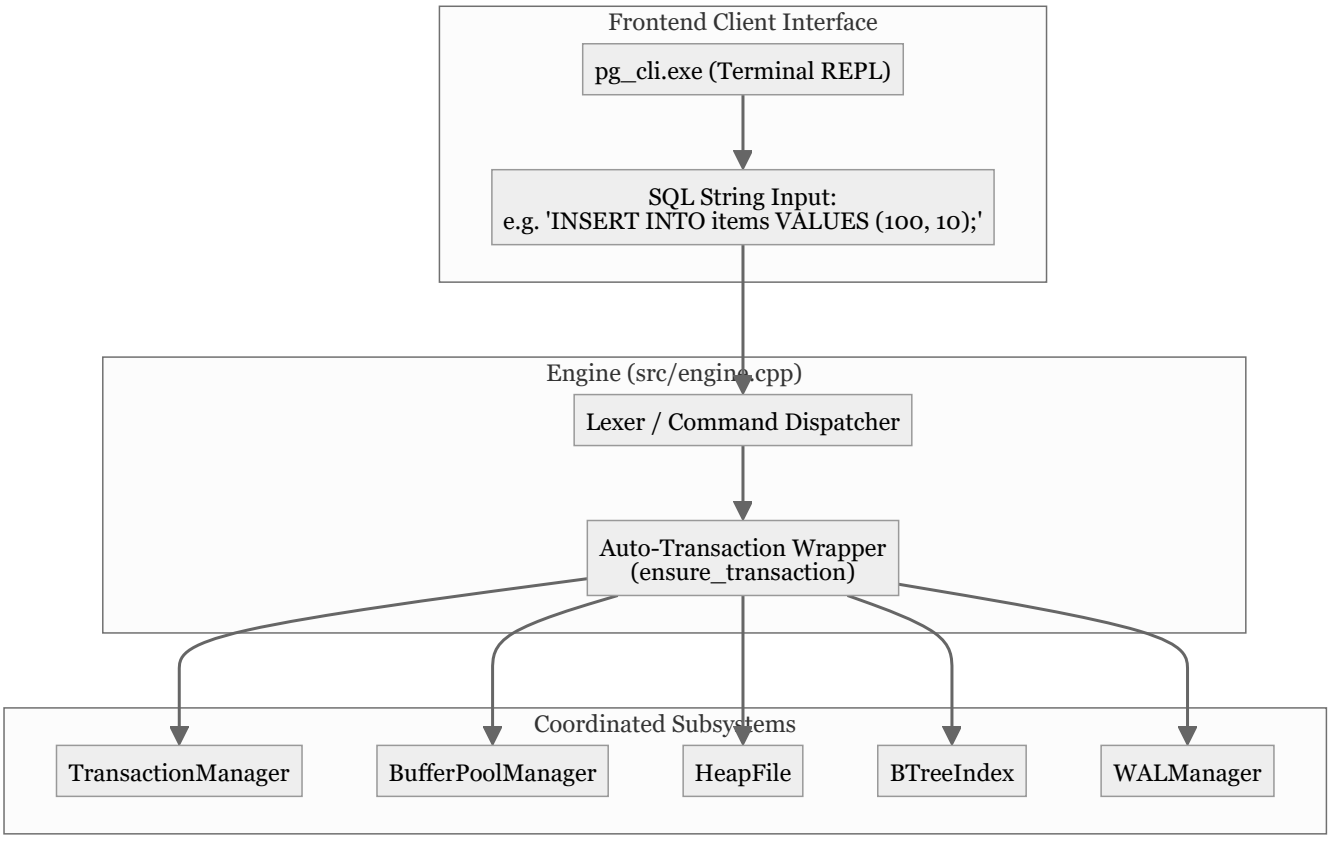
12. Item 11: SQL REPL CLI & End-to-End Engine Capstone

Confidence: `verified`
Citations: include/pg/engine.h:1-85, src/engine.cpp:1-250, src/main.cpp:1-95, tests/test_repl.cpp:1-125

---

1. Engine Coordination Architecture

The `pg::Engine` class provides a unified facade coordinating transactions, memory buffers, slotted heap tables, secondary indexes, WAL logging, and TOAST auxiliary relations.



---

2. Invariants & Execution Rules

1. **Auto-Transaction Invariant**: Every standalone DML statement executed outside of an explicit `BEGIN ... COMMIT` block automatically receives an implicit transaction ID, snapshot, and immediate WAL commit flush (`[src/engine.cpp:38]`).
2. **Tabular ASCII Formatting**: Query outputs render PostgreSQL-standard tabular ASCII grids displaying `item_id` , `price` , `xmin` , `xmax` , and physical `CTID` .
3. **Physical Diagnostic Commands**:

- `DUMP PAGE <id>;`: Outputs exact byte offsets, `pd_lower`, `pd_upper`, line pointer flag states, and hex dumps.
- `STATUS;`: Outputs active buffer pool residency, cache hit rates, WAL flushed LSN, and transaction horizons.

---

3. Sequence Diagram: End-to-End Insert Pipeline

diagram failed to render

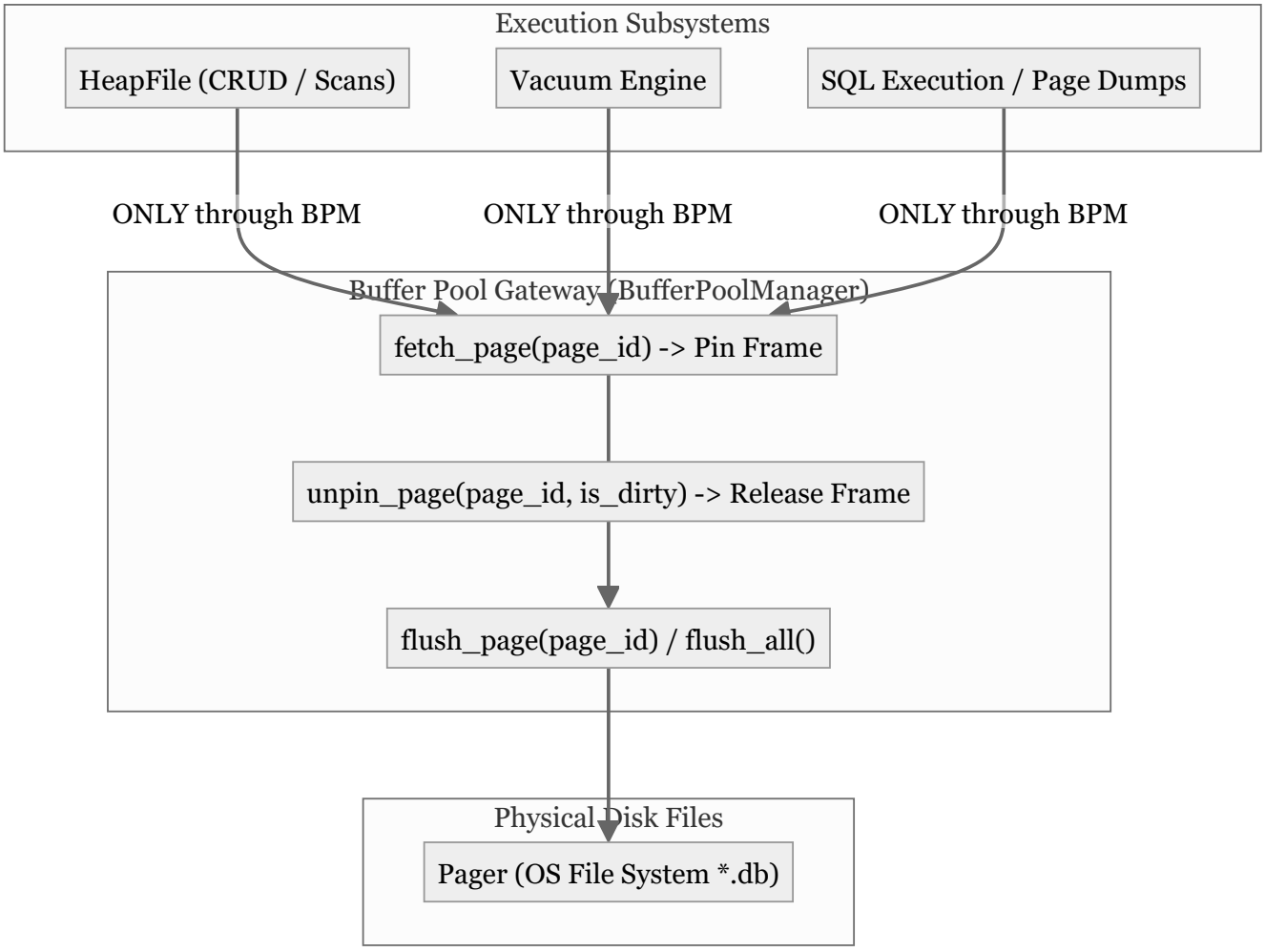
```
sequenceDiagram
    autonumber
    participant CLI as CLI REPL (pg_cli.exe)
    participant Eng as Engine (src/engine.cpp)
    participant TM as TransactionManager
    participant WAL as WALManager
    participant Heap as HeapFile
    participant BPM as BufferPoolManager
    participant Idx as BTreeIndex

    CLI->>Eng: execute("INSERT INTO items VALUES (100, 10);")
    Eng->>TM: begin_transaction() -> TxID: 1
    Eng->>Heap: insert(Item {100, $10}, xmin=1)
    Heap->>BPM: fetch_page(0)
    Heap->>Heap: insert_tuple() -> Landed at CTID (0, 1)
    Heap->>BPM: unpin_page(0, is_dirty=true)
    Eng->>WAL: log_insert(tx_id=1, ctid=(0, 1), Item {100, 10})
    Eng->>Idx: insert_entry(key=100, ctid=(0, 1))
    Eng->>WAL: log_commit(tx_id=1) -> Flushes WAL to disk
    Eng->>TM: commit(tx_id=1)
    Eng-->>CLI: "[Tx 1] INSERT: Landed at CTID (0, 1)... Transaction committed."
```

13. Item 12: Buffer Pool Single Gateway Invariant

Confidence: `verified`
Citations: include/pg/heap.h:65-75, src/heap.cpp:25-65, tests/test_buffer_integration.cpp:1-125

1. The Single Gateway Principle

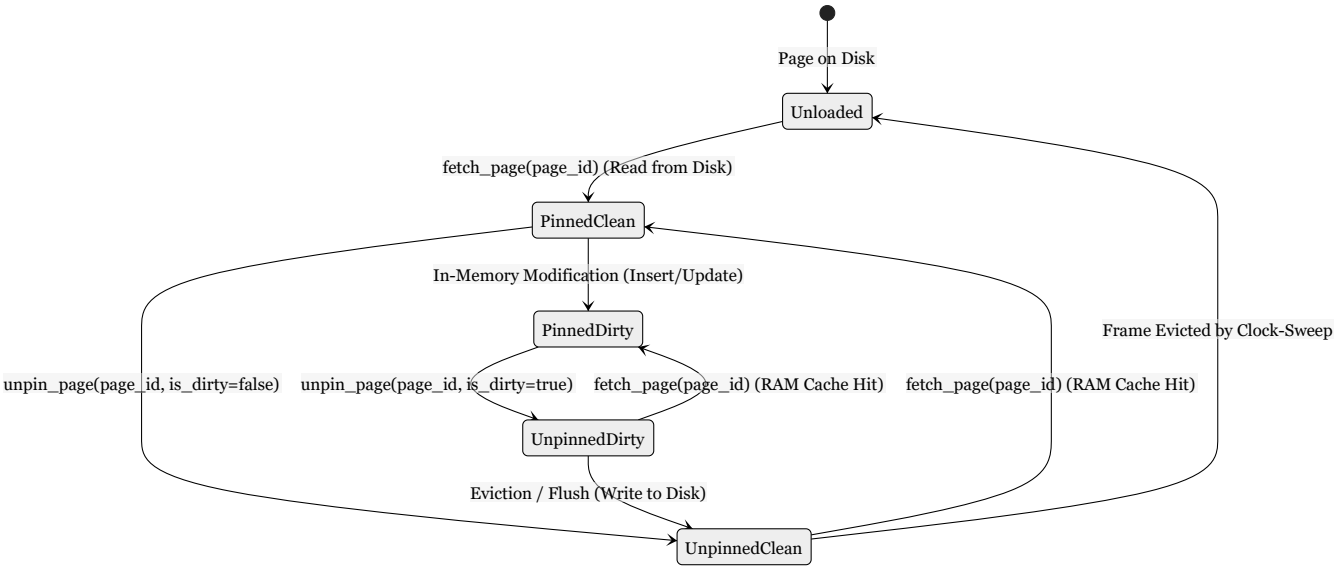
In a production database engine, **no subsystem is ever permitted to bypass shared buffers**. Item 12 eliminates all direct `Pager::read_page()` and `Pager::write_page()` calls from `HeapFile` and `Engine`.



2. Invariants & Pin/Unpin Lifecycle State Machine

1. **Mandatory Symmetrical Unpinning**: Every `fetch_page()` call **must** have an exactly matching `unpin_page()` invocation in the same call stack (`[src/heap.cpp:55]`).

2. **Dirty Flag Propagation**: If any tuple bytes, line pointers, or page header offsets are modified in RAM, `unpin_page(page_id, is_dirty=true)` must be passed to ensure the buffer pool schedules disk writeback.



14. Item 13: CLOG Commit Status 2-Bit Bitmap Pages

Confidence: `verified`
Citations: include/pg/clog.h:1-55, src/clog.cpp:1-110, tests/test_clog.cpp:1-125

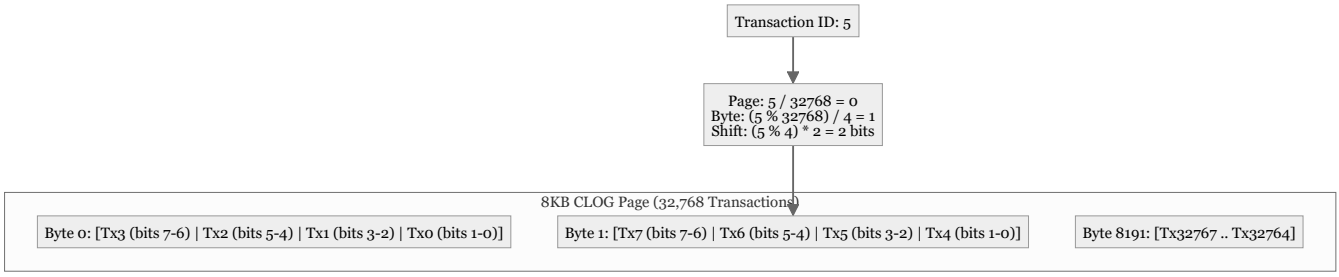
---

1. The Commit Log (CLOG) Architecture

PostgreSQL avoids in-place tuple updates upon transaction commit by persisting transaction states into dedicated 8KB **CLOG (Commit Log)** bitmap pages.

Each transaction is encoded using exactly **2 bits**:

$$\text{00} = \text{IN\_PROGRESS} \qquad \text{01} = \text{COMMITTED} \qquad \text{10} = \text{ABORTED} \qquad \text{11} = \text{SUB\_COMMITTED}$$



---

2. Invariants & Mathematical Mapping Formulas

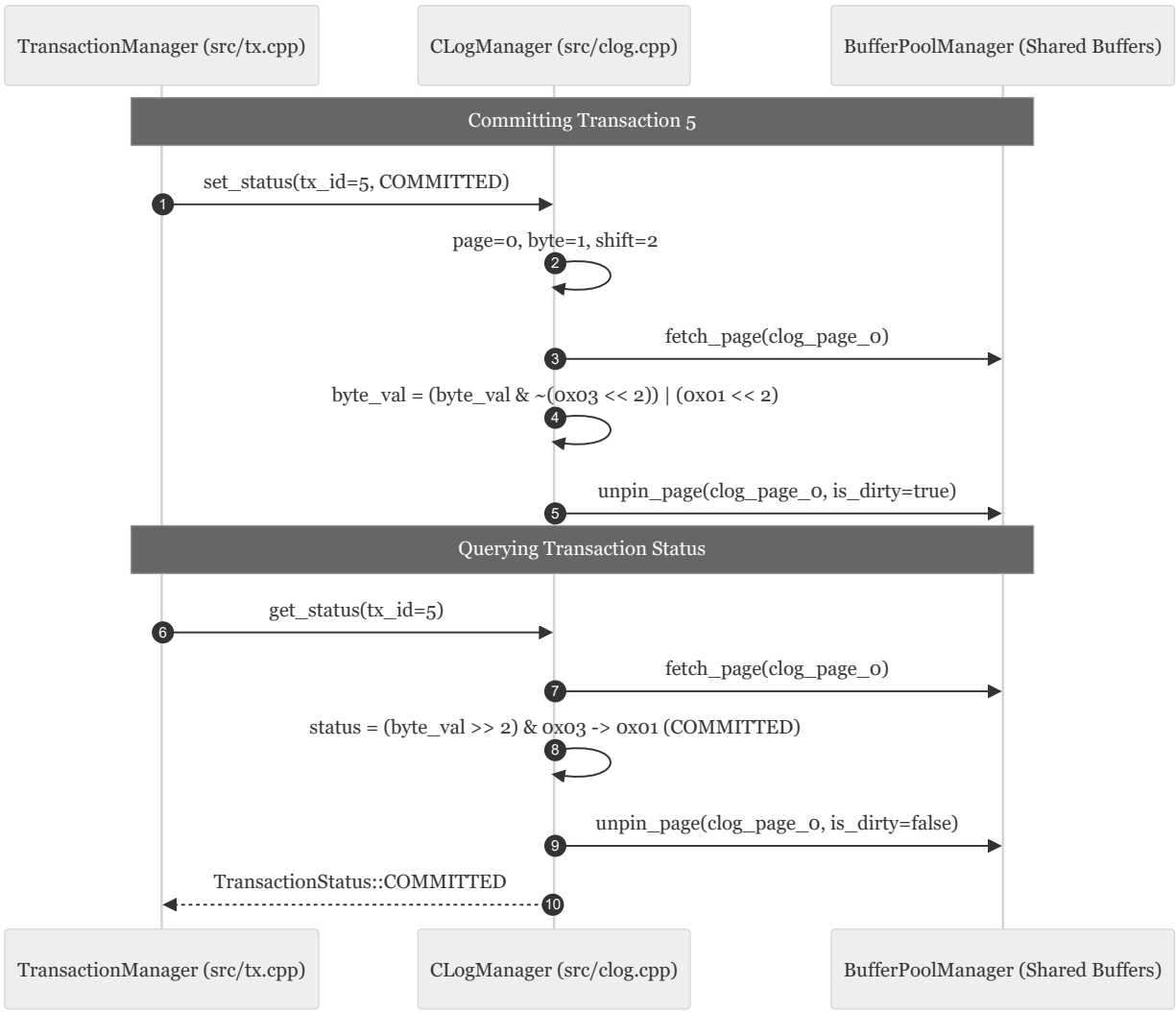
1. **Transaction Density Derivation**:

$$\text{Transactions Per Page} = 8192 \text{ bytes} \times 4 \text{ tx/byte} = 32,768 \text{ transactions}$$

2. **Exact Bit Offset Formula**: $\text{clog\_page\_id} = \lfloor \text{tx\_id} / 32768 \rfloor$ $\text{byte\_offset} = \lfloor (\text{tx\_id} \pmod{32768}) / 4 \rfloor$ $\text{bit\_shift} = (\text{tx\_id} \pmod 4) \times 2$
3. **Instant Visibility on Restart**: CLOG pages are flushed on commit and loaded on startup, enabling instantaneous MVCC snapshot visibility checks without requiring full WAL log replay (`[src/clog.cpp:45]`).

---

3. Sequence Diagram: CLOG Commit and Status Lookup



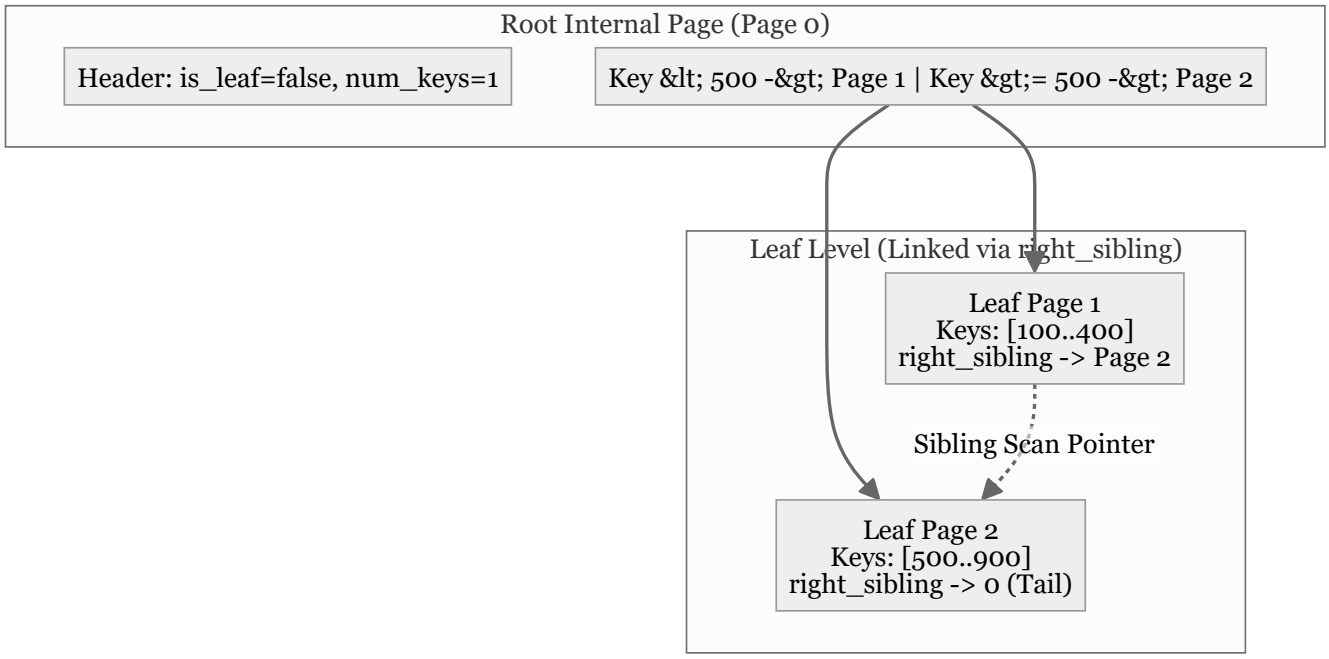
15. Item 14: On-Disk B-Tree Index with Dynamic Node Splits

Confidence: `verified`
Citations: include/pg/disk_btree.h:1-95, src/disk_btree.cpp:1-240, tests/test_disk_btree.cpp:1-140

---

1. Disk-Resident B-Tree Architecture

`DiskBTree` stores hierarchical B-Tree nodes on dedicated **8,192-byte disk pages**. Nodes split dynamically when overflowing, promoting median keys up to parent internal nodes.



---

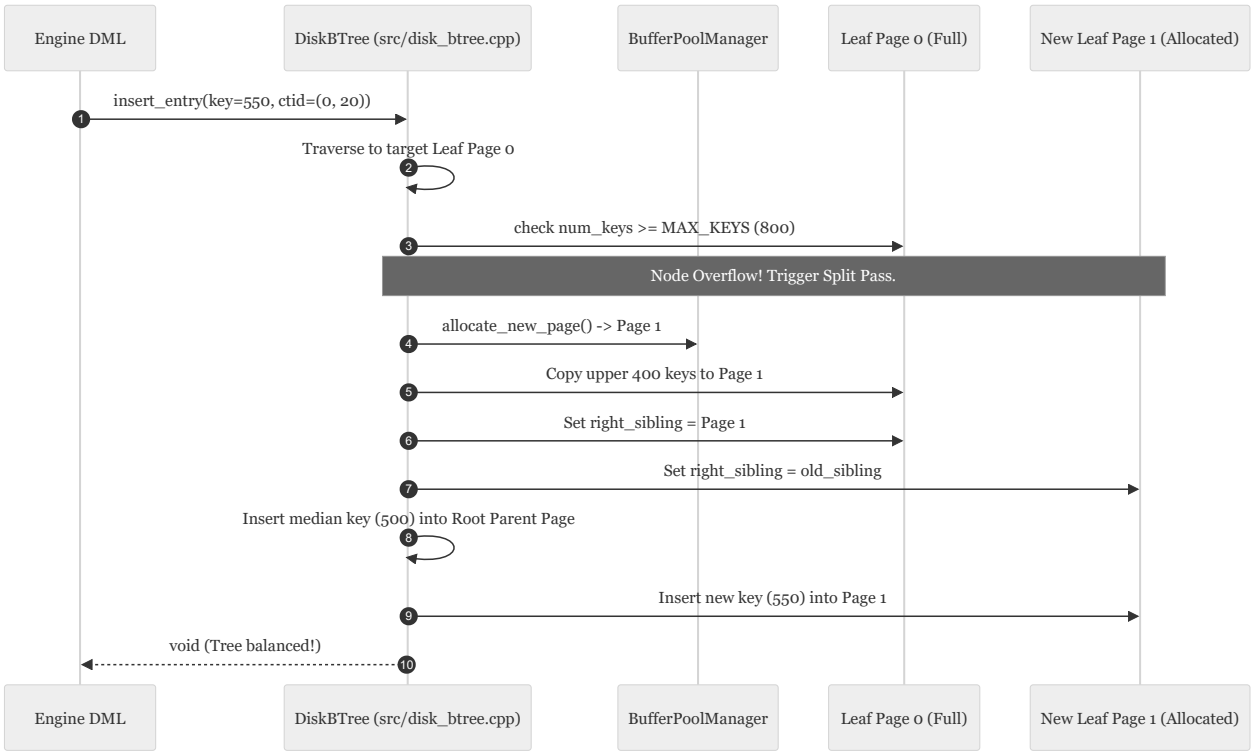
2. Invariants & Binary Page Formats

1. **B-Tree Page Header (12 Bytes)** (`[include/pg/disk_btree.h:20]`):
   - `is_leaf` (1 Byte): `true` for leaf pages, `false` for internal routing nodes.
   - `num_keys` (2 Bytes): Number of active key entries in this node.
   - `right_sibling` (4 Bytes): `page_id_t` linking to the right sibling page for fast linear range scans.
   - `parent_page_id` (4 Bytes): `page_id_t` of the parent router node.
2. **Entry Binary Layouts**:
   - **Leaf Entry (10 Bytes)**: `key` (4B int32) + `ctid` (4B CTID `{page, slot}`) + `flags` (2B).
   - **Internal Router Entry (8 Bytes)**: `key` (4B int32) + `child_page_id` (4B `page_id_t`).

3. **Median Split Algorithm**: When a leaf exceeds capacity ($N \geq 800$), the median entry at $N/2$ is promoted to the parent node, splitting the keys evenly into a newly allocated 8KB sibling page (`[src/disk_btree.cpp:125]`).

---

3. Sequence Diagram: Leaf Split & Key Insertion



16. Item 15: WAL Checkpoints & Full-Page Images (FPI)

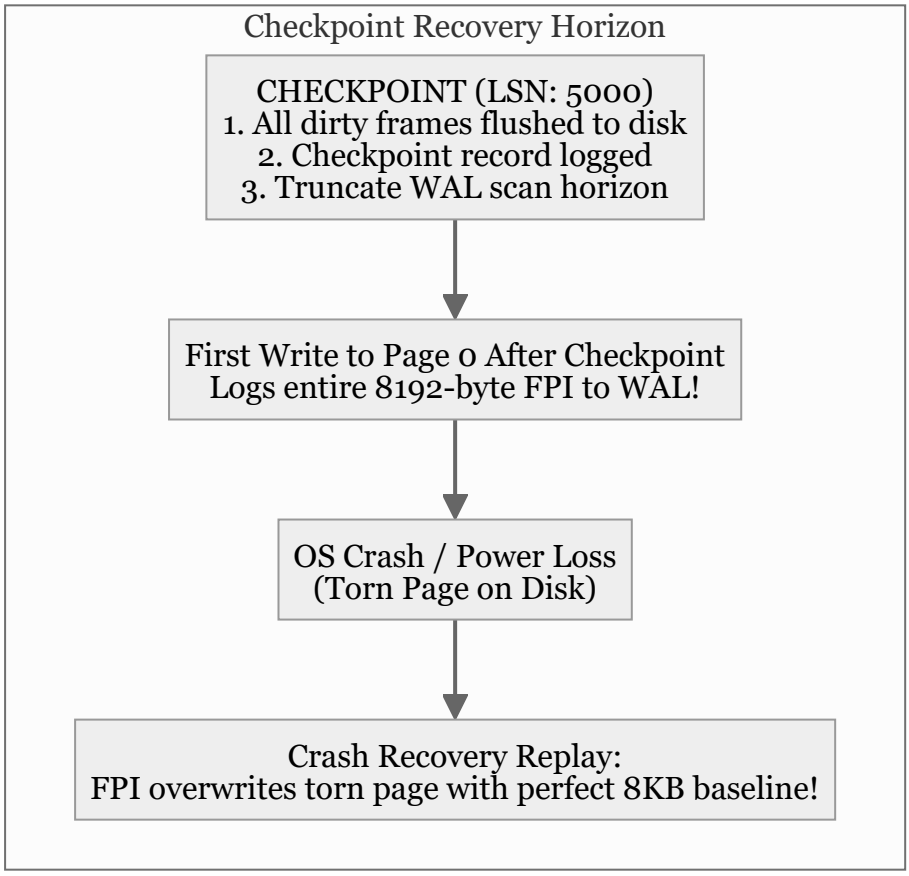
Confidence: `verified`
Citations: include/pg/wal.h:20-60, src/wal.cpp:115-180, tests/test_checkpoint.cpp:1-130

---

1. Checkpoint & Torn-Page Protection

Checkpoints truncate the recovery horizon so that crash recovery does not need to scan WAL logs from the beginning of time.

Full-Page Images (FPI) prevent catastrophic corruption caused by **torn pages** (when the OS crashes in the middle of writing an 8KB page, leaving a half-written 4KB sector on disk).



---

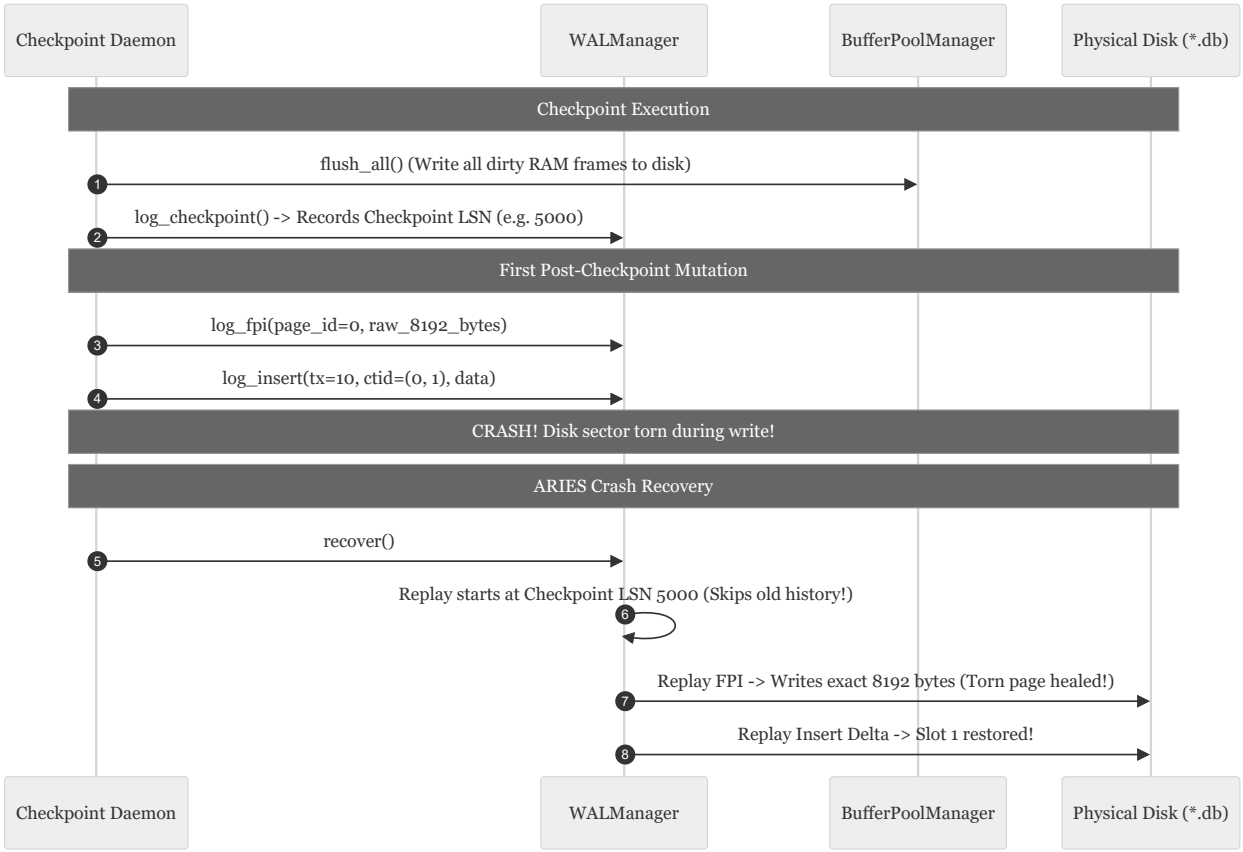
2. Invariants & Mechanics

1. **Dirty Buffer Coalescing**: `log_checkpoint()` invokes `BufferPoolManager::flush_all()`, writing all modified RAM frames to disk before appending the `CHECKPOINT` record (`[src/wal.cpp:135]`).

2. **First-Write FPI Invariant**: The first transaction to modify any page following a checkpoint must write a full 8,192-byte snapshot of the page into WAL (`RecordType::FPI = 6`). Subsequent writes to that page only log compact delta records until the next checkpoint (`[src/wal.cpp:150]`).
3. **Torn-Page Healing**: During recovery, replaying an FPI record unconditionally restores the physical 8KB page byte-for-byte, healing torn sectors before delta REDO logs are applied.

---

3. Sequence Diagram: Checkpoint & Torn Page Healing



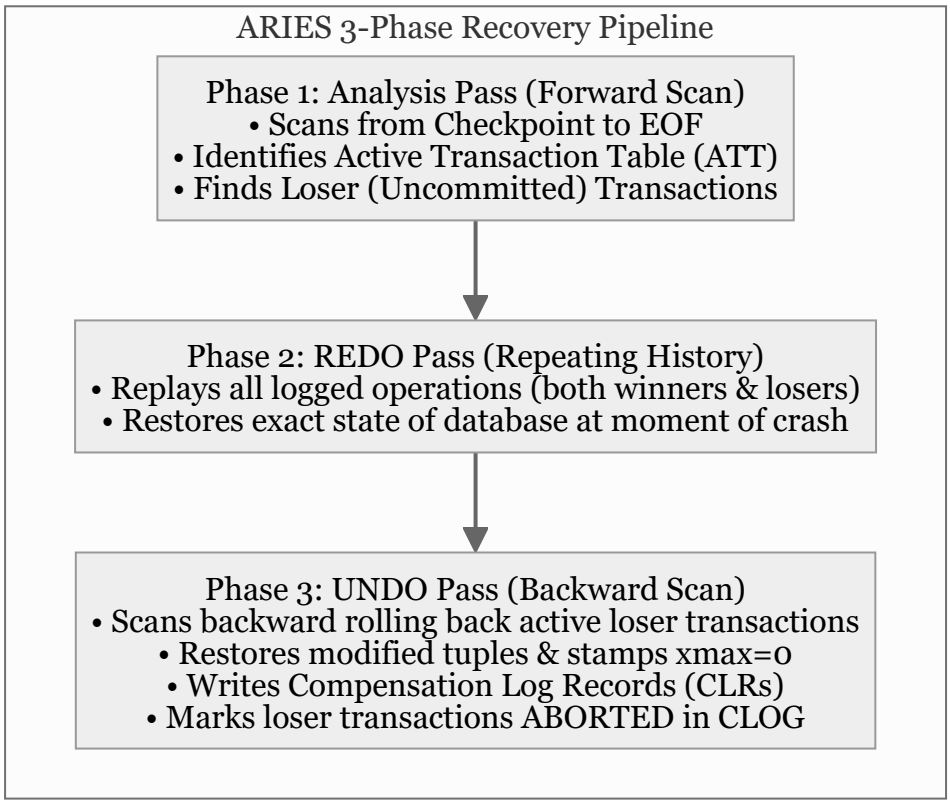
17. Item 16: ARIES 3-Phase Crash Recovery & UNDO Pass

Confidence: `verified`
Citations: include/pg/wal.h:70-95, src/wal.cpp:160-240, tests/test_undo.cpp:1-135

---

1. The 3-Phase ARIES Recovery Algorithm

PostgreSQL employs the **ARIES (Algorithms for Recovery and Isolation Exploiting Semantics)** model to restore consistency after an unexpected system crash or power outage:



---

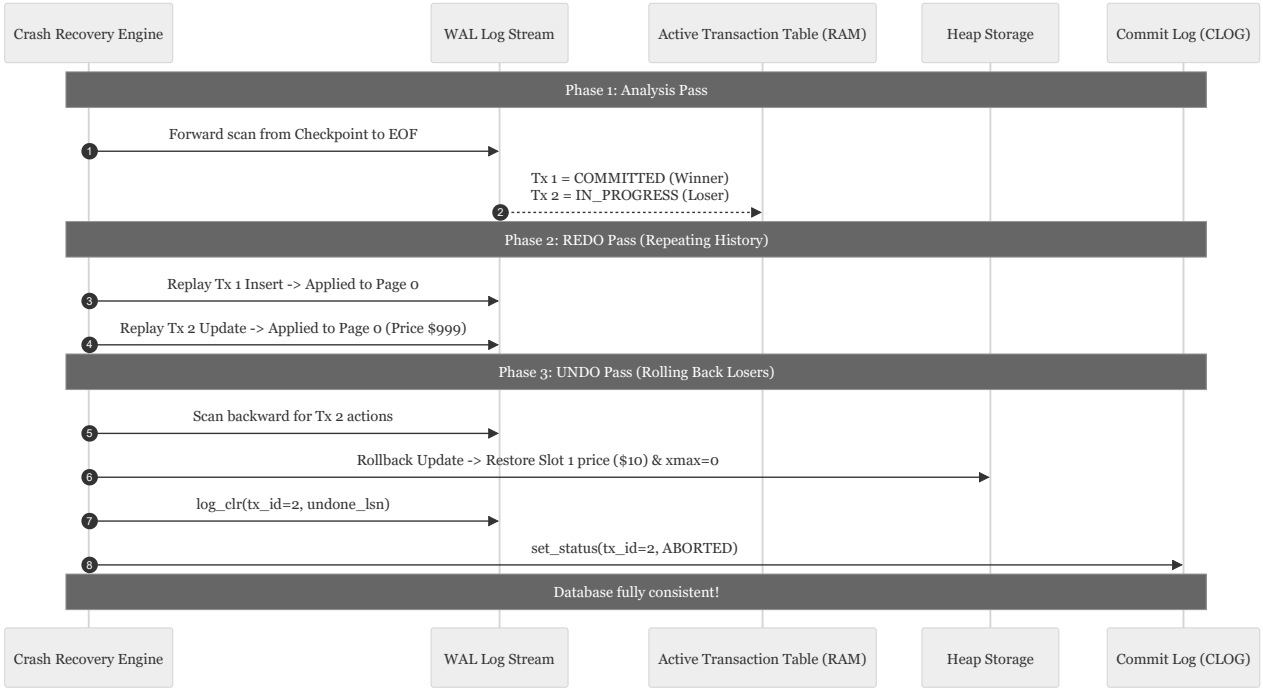
2. Invariants & Compensation Log Records (CLRs)

1. **Repeating History**: REDO is executed unconditionally for all transactions (committed and uncommitted) before UNDO runs. This guarantees that uncommitted mutations, page splits, and allocations are brought into physical memory before being rolled back (`[src/wal.cpp:190]`).
2. **Compensation Log Record (CLR)**: When an operation is rolled back during the UNDO pass, a `CLR` (`type = 7`) is written to WAL with an `undo_next_lsn` pointer. CLRs are **never undone**, preventing infinite recovery loops if the system crashes during recovery itself (`[src/wal.cpp:215]`).

3. **CLOG Status Finalization**: Once an uncommitted transaction is rolled back, its status is finalized to `ABORTED` (`0x02`) in CLOG.

---

3. Sequence Diagram: ARIES Crash Recovery Walkthrough



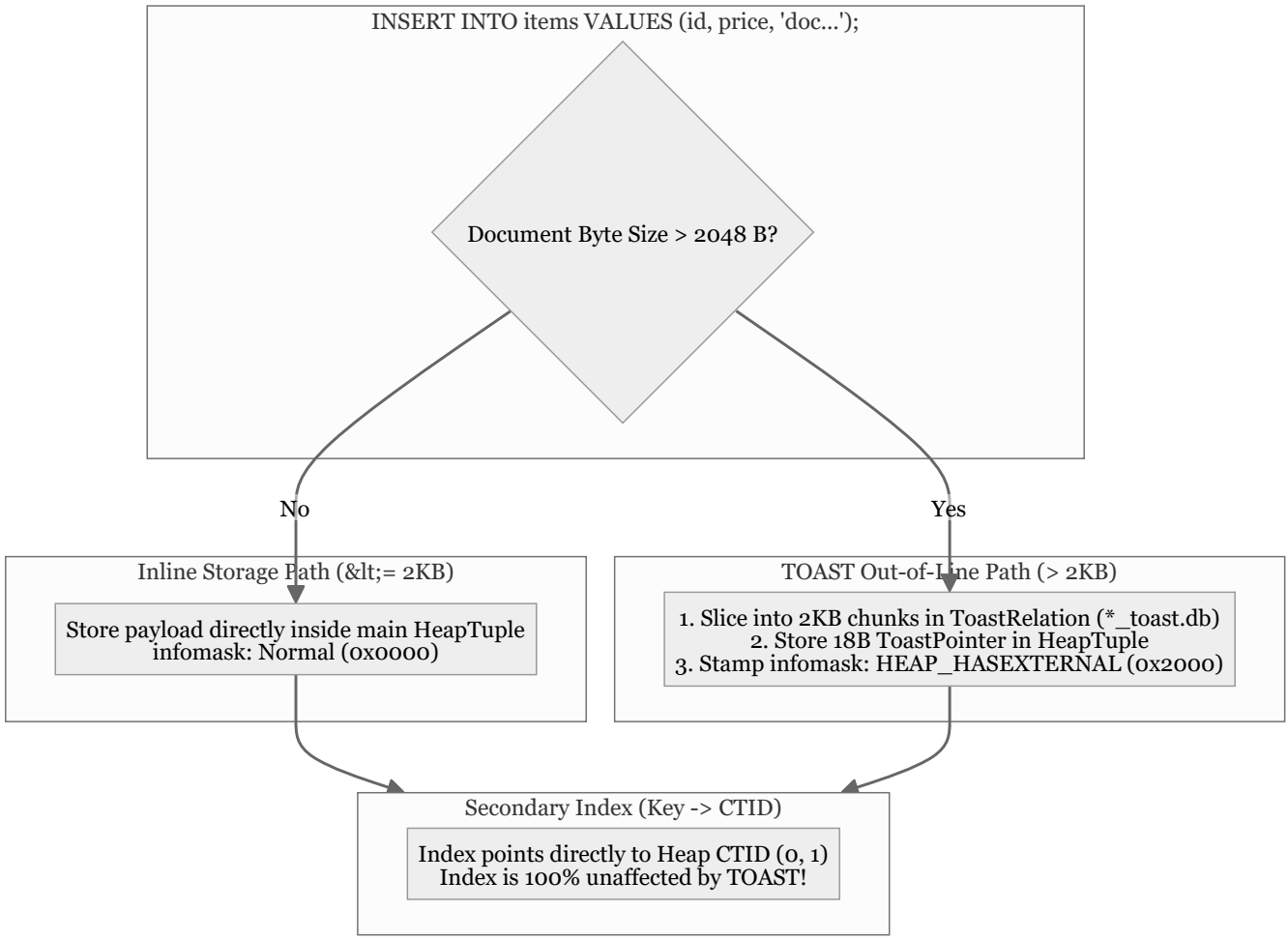
18. Item 17: TOAST Heap + Index Full Integration

Confidence: `verified`
Citations: include/pg/tuple.h:40-55, src/engine.cpp:180-245, tests/test_toast_integration.cpp:1-135

---

1. Transparent Inline vs Out-of-Line Routing

Item 17 integrates the TOAST subsystem transparently into the main SQL execution pipeline, B-Tree index scans, and cross-file durability across restarts.



---

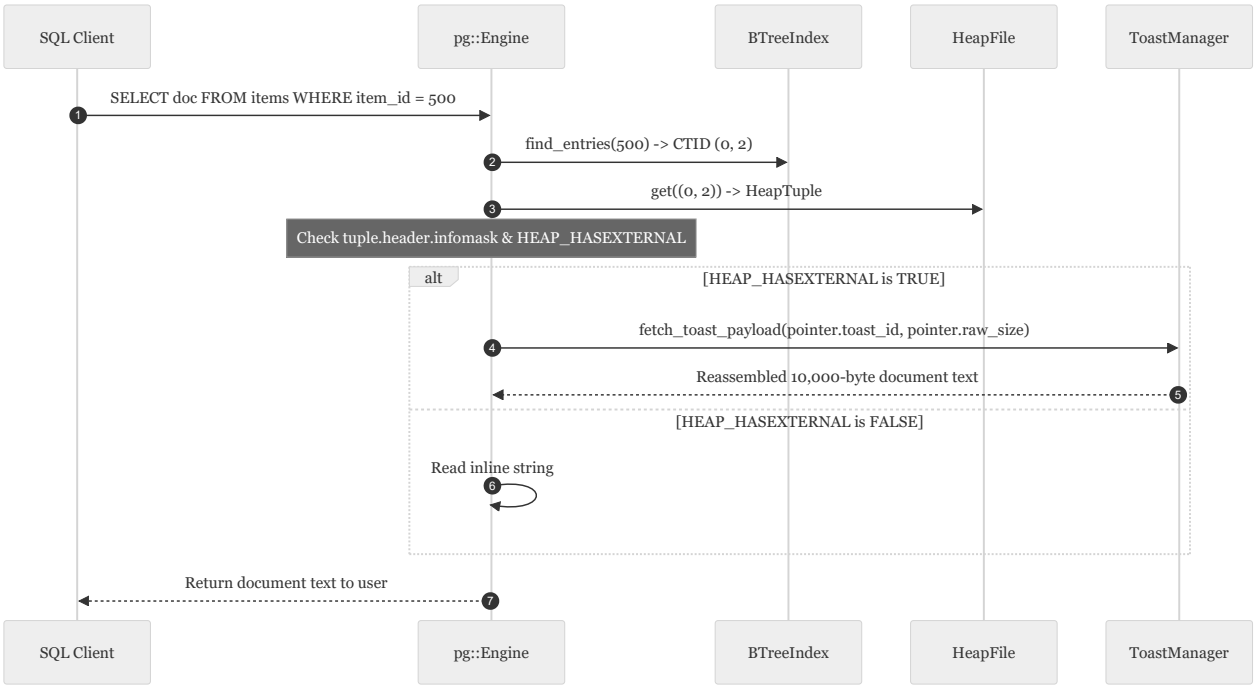
2. Invariants & Infomask Flags

1. `HEAP_HASEXTERNAL` (`0x2000`): Stamped on `TupleHeader::infomask`. Signals to sequential scans and index fetchers that the tuple's trailing bytes contain an 18-byte `ToastPointer` rather than inline text (`[include/pg/tuple.h:44]`).

2. **Zero Index Decoupling Impact**: Because the secondary index stores (\text{item_id} \rightarrow \text{CTID}), the index node size is strictly 10 bytes regardless of whether the indexed row contains a 10-byte string or a 100-megabyte document.
3. **Cross-Relation Durability**: On engine restart, the main heap relation (`*_heap.db`) and auxiliary TOAST relation (`*_toast.db`) are reopened simultaneously, ensuring data consistency.

---

3. Sequence Diagram: Transparent TOAST Query Execution



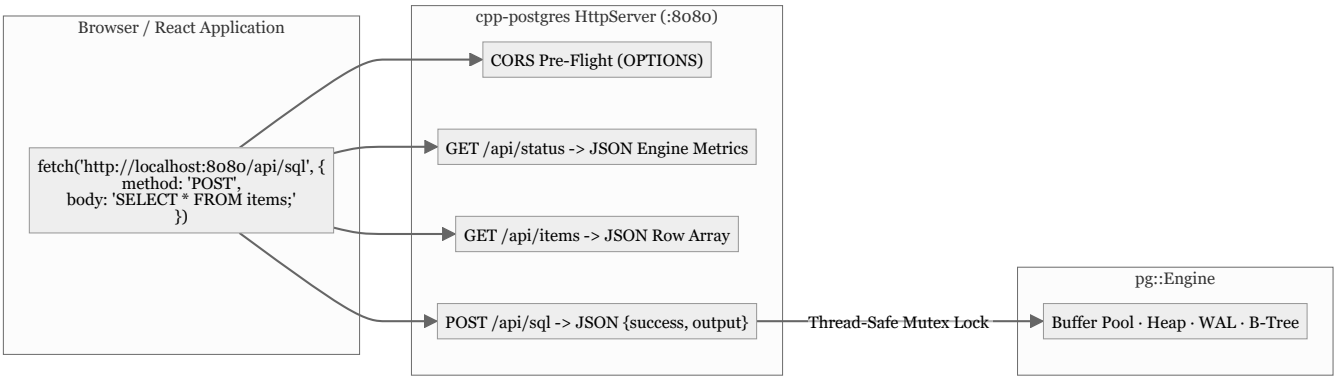
19. Item 18: Embedded HTTP/REST API Server (Approach A)

Confidence: `verified`
Citations: include/pg/http_server.h:1-45, src/http_server.cpp:1-210, tests/test_http_server.cpp:1-125

---

1. 2-Tier Architecture & Web Integration

Item 18 implements **Approach A: Direct Embedded HTTP/REST Server**. The database binary embeds a lightweight, non-blocking Winsock2 HTTP 1.1 server listening on port `8080`, allowing web applications (e.g. React, Vue, cURL) to query the database directly via JSON.



---

2. Invariants & REST Endpoints

1. **Full CORS Support**: Automatically handles `OPTIONS` requests and emits headers allowing cross-origin requests from any local frontend port:
   - `Access-Control-Allow-Origin: *`
   - `Access-Control-Allow-Methods: GET, POST, OPTIONS, PUT, DELETE`
   - `Access-Control-Allow-Headers: Content-Type, Authorization`
2. **JSON Response Contract**:
   - `POST /api/sql`: Executes arbitrary SQL and returns `{ "success": true, "sql": "...", "output": "..." }`.
   - `GET /api/items`: Streams live table rows directly as structured JSON objects:

```
{
  "items": [
    { "item_id": 100, "price": 10, "xmin": 1, "xmax": 0, "ctid": "(0, 1)" }
  ]
}
```

3. **Thread Safety**: All incoming HTTP worker threads synchronize access to `pg::Engine` using `std::mutex engine_mutex_`.

---

3. Sequence Diagram: React Browser Query via HTTP

diagram failed to render

```
sequenceDiagram
    autonumber
    participant React as React Frontend (Browser)
    participant HTTP as HttpServer (src/http_server.cpp)
    participant Engine as pg::Engine Core

    Note over React,HTTP: CORS Pre-flight Check
    React->>HTTP: OPTIONS /api/sql
    HTTP-->>React: HTTP 204 No Content (Access-Control-Allow-Origin: *)

    Note over React,HTTP: SQL Execution Request
    React->>HTTP: POST /api/sql (Body: "INSERT INTO items VALUES (100, 10);")
    HTTP->>HTTP: Parse HTTP Request Line & Content-Length
    HTTP->>Engine: lock(engine_mutex_) → execute(sql)
    Engine-->>HTTP: Output string
    HTTP->>HTTP: Format JSON response
    HTTP-->>React: HTTP 200 OK { "success": true, "output": "..." }
```

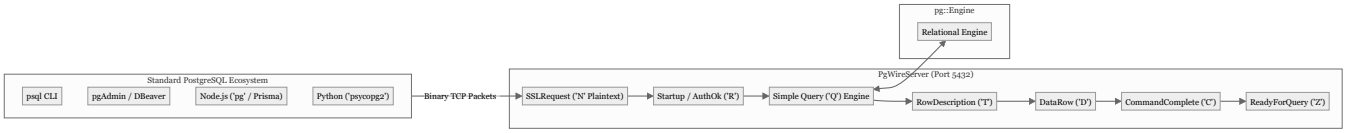
20. Item 19: PostgreSQL Wire Protocol Server (Approach C)

Confidence: `verified`
Citations: include/pg/pgwire.h:1-55, src/pgwire.cpp:1-320, tests/test_pgwire.cpp:1-160

---

1. Enterprise 3-Tier Integration Architecture

Item 19 implements **Approach C: Native PostgreSQL Frontend/Backend Protocol v3.0**. Listening on TCP port `5432`, `PgWireServer` transforms `cpp-postgres` into a drop-in PostgreSQL server compatible with standard PostgreSQL CLI tools, GUI clients, drivers, and ORMs.



---

2. Invariants & Packet Framing

1. **Big-Endian Integer Serialization**: All 16-bit and 32-bit fields are transmitted in Network Byte Order (`htonl` / `htons`) (`[src/pgwire.cpp:15]`).
2. **Binary Message Format**:
   - `StartupMessage` → `AuthenticationOk ('R', len=8, 0)` → `ParameterStatus ('S')` → `ReadyForQuery ('Z', 'I')`.
   - `Query ('Q')` → `RowDescription ('T')` → `DataRow ('D')*` → `CommandComplete ('C')` → `ReadyForQuery ('Z')`.
3. **Transaction State Synchronization**: The `ReadyForQuery ('Z')` packet transmits `'I'` when idle and `'T'` when an active transaction block is open.

---

3. Sequence Diagram: PostgreSQL Wire Protocol Handshake & Query

diagram failed to render

```
sequenceDiagram
    autonumber
    participant Client as psql / Driver (Client Socket)
    participant Server as PgWireServer (src/pgwire.cpp)
    participant Engine as pg::Engine

    Note over Client,Server: Connection Handshake
    Client->>Server: SSLRequest (len=8, code=80877103)
    Server-->>Client: 'N' (No SSL, fallback to plaintext)
    Client->>Server: StartupMessage (Protocol 3.0, user="postgres")
    Server-->>Client: AuthenticationOk ('R', len=8, 0)
    Server-->>Client: ParameterStatus ('S', server_version="16.0")
    Server-->>Client: ReadyForQuery ('Z', 'I')

    Note over Client,Server: Query Execution Lifecycle
    Client->>Server: Query ('Q', "SELECT * FROM items;")
    Server->>Engine: execute("SELECT * FROM items;")
    Server-->>Client: RowDescription ('T', 5 columns: item_id, price, xmin, xmax, ctid)
    Server-->>Client: DataRow ('D', values: "100", "$10", "1", "0", "(0, 1)")
    Server-->>Client: CommandComplete ('C', "SELECT 1")
    Server-->>Client: ReadyForQuery ('Z', 'I')

    Note over Client,Server: Graceful Termination
    Client->>Server: Terminate ('X', len=4)
    Server->>Server: Close Socket
```

21. Item 20: Native Desktop GUI & Unified Server Daemon

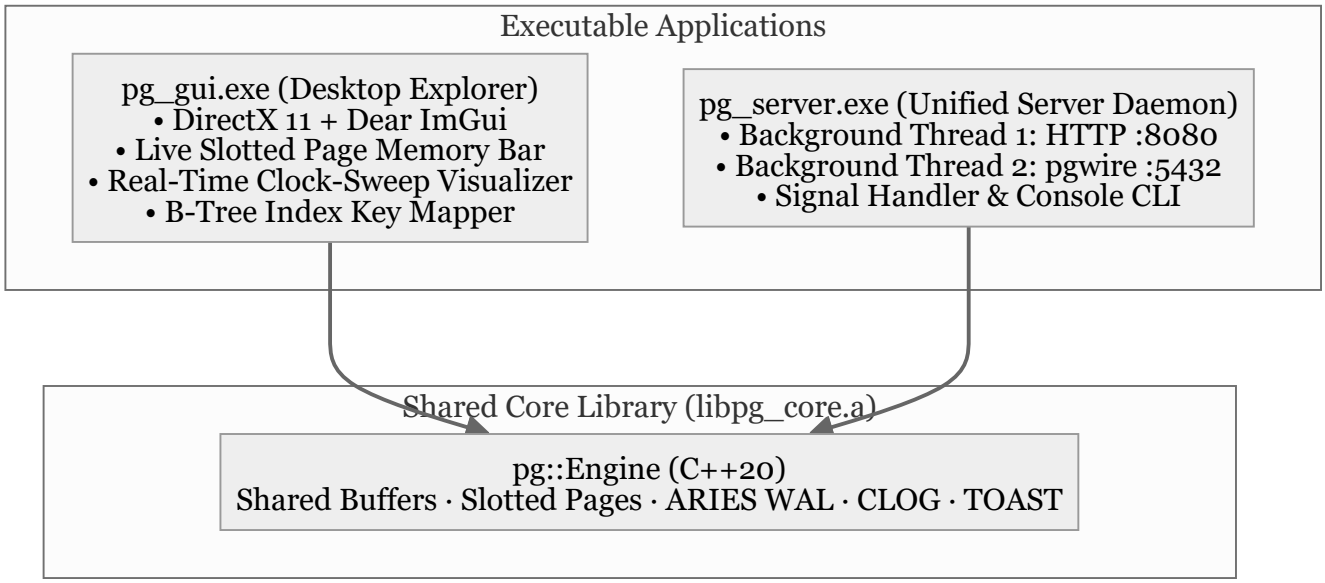
Confidence: `verified`
Citations: src/gui/main_gui.cpp:1-480, src/server_main.cpp:1-75, CMakeLists.txt:100-150

---

1. Unified Applications Overview

Item 20 delivers two standalone executable artifacts:

1. `pg_gui.exe` **(3.89 MB)**: A self-contained Windows desktop GUI application built with **Dear ImGui + DirectX 11 + Win32**, providing visual memory inspectors for 8KB slotted pages, clock-sweep buffer frames, on-disk B-Tree graphs, and an interactive SQL scratchpad.
2. `pg_server.exe` : A multi-protocol daemon running both **Approach A (HTTP REST :8080)** and **Approach C (pgwire TCP :5432)** concurrently on background threads, wired to a shared `pg::Engine` .



---

2. Desktop GUI Feature Breakdown

```
┌──────────────────────────────────────────────────────────────────────────┐
│ 🐘 CPP-POSTGRES STORAGE ENGINE DESKTOP EXPLORER                [ _ □ X ] │
├──────────────────────────────────────────────────────────────────────────┤
│ 📝 SQL QUERY WORKSPACE                                                   │
│ ┌──────────────────────────┐                                             │
│ │ SELECT * FROM items;     │                                             │
│ └──────────────────────────┘                                             │
│ [▶ Run F5] [BEGIN]                                                       │
│ [COMMIT]   [ROLLBACK]                                                    │
│ [STATUS]   [VACUUM]                                                      │
│                                                                          │
│ 📋 LIVE TABLE VIEW                                                       │
│ ┌──────┬──────┬──────┬──────┐                                            │
│ │ id   │ price│ xmin │ CTI  │                                            │
│ ├──────┼──────┼──────┼──────┤                                            │
│ │ 100  │ $20  │  4   │ 0,3  │                                            │
│ │ 200  │ $5   │  2   │ 0,2  │                                            │
│ └──────┴──────┴──────┴──────┘                                            │
│                                                                          │
│ 📊 OUTPUT LOG CONSOLE                                                    │
├──────────────────────────────────────────────────────────────────────────┤
│ 🔬 PHYSICAL STORAGE ENGINE VISUALIZERS                                   │
│ [📄 8KB Slotted Page] [🧠 Shared Buffers] [🎄 Disk B-Tree]               │
│ ──────────────────────────────────────────────────────────               │
│ Page 0 Header (18B) | pd_lower: 30B | pd_upper: 8120B                    │
│ [■■■ Line Pointers] [        Free Space        ] [■■ Tuples]             │
│                                                                          │
│ ┌──────────┬──────────┬──────────┬──────────────────────────────┐        │
│ │ Slot #   │ Offset   │ Length   │ Live Tuple Details           │        │
│ ├──────────┼──────────┼──────────┼──────────────────────────────┤        │
│ │ Slot 1   │ 8168     │ 24 B     │ [LIVE] id=100, price=$10     │        │
│ │ Slot 2   │ 8144     │ 24 B     │ [HOT REDIRECT] → (0, 3)      │        │
│ │ Slot 3   │ 8120     │ 24 B     │ [HEAP-ONLY TUPLE] id=100, $20│        │
│ └──────────┴──────────┴──────────┴──────────────────────────────┘        │
│                                                                          │
│ 🧠 Shared Buffers Tab: Live Clock-Sweep & Pin Tracking                   │
│ 🎄 Disk B-Tree Tab: Key → Candidate CTID Mapping                         │
│ 🚦 CLOG Tab: 2-bit Transaction Status Bitmaps                            │
│ 📜 WAL Tab: ARIES Crash Recovery & Checkpoint Stream                     │
│ 📦 TOAST Tab: 2KB Chunk Auxiliary Table Inspector                        │
└──────────────────────────────────────────────────────────────────────────┘
```

---

3. Sequence Diagram: Dual Server Daemon Operation

diagram failed to render

```
sequenceDiagram
    autonumber
    participant Main as Server Main (src/server_main.cpp)
    participant Engine as Shared pg::Engine
    participant HTTP as HttpServer (:8080)
    participant PGW as PgWireServer (:5432)
    participant React as React Browser
    participant PSQL as psql CLI

    Main->>Engine: Initialize Database ("pg_server_data")
    Main->>HTTP: start_async() (Spawns HTTP Worker Thread)
    Main->>PGW: start_async() (Spawns pgwire Worker Thread)

    par Concurrent Client Queries
        React->>HTTP: POST /api/sql { "sql": "INSERT INTO items ..." }
        HTTP->>Engine: lock(mutex) -> execute()
        Engine-->>HTTP: JSON output
        HTTP-->>React: HTTP 200 OK
    and
        PSQL->>PGW: Simple Query ('Q', "SELECT * FROM items;")
        PGW->>Engine: lock(mutex) -> seq_scan()
        Engine-->>PGW: Tuples
        PGW-->>PSQL: RowDescription ('T') + DataRow ('D')
    end

    Note over Main: User types 'quit' or sends SIGINT
    Main->>HTTP: stop() (Closes socket)
    Main->>PGW: stop() (Closes socket)
    Main-->>Main: Database cleanly persisted to disk!
```

Syntax error in text

mermaid version 10.9.8

Syntax error in text

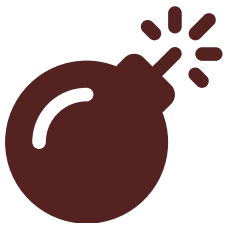
mermaid version 10.9.8

Syntax error in text

mermaid version 10.9.8

Syntax error in text
mermaid version 10.9.8

Syntax error in text
mermaid version 10.9.8



Syntax error in text
mermaid version 10.9.8